



Printers' Spareparts and Maintenance System Documentation



1. Introduction

1.1 Project Overview

The rapid growth of e-commerce and digital services has significantly transformed the way businesses operate and interact with customers. In the field of hardware maintenance and spare parts distribution, there is a growing need for integrated systems that combine product sales with maintenance services in a single platform.

This project presents the design and development of a web-based system for selling printer spare parts and managing maintenance appointments. The system allows users to browse and purchase products, as well as request maintenance services for their devices. It also provides administrative functionalities for managing inventory, users, and service operations.

1.2 Problem Statement

Traditional systems for selling spare parts and handling maintenance requests are often inefficient and fragmented. Customers may face difficulties in finding the required parts, tracking their orders, or scheduling maintenance services. On the other hand, businesses struggle with managing inventory, coordinating technicians, and maintaining accurate records.

There is a need for a unified system that simplifies these processes by providing an organized, user-friendly, and efficient platform for both customers and administrators.

1.3 Project Objectives

- To develop an online platform for selling printer spare parts
- To provide a system for booking and managing maintenance appointments
- To implement role-based access for different users
- To enable efficient inventory and product management
- To ensure secure and reliable payment processing
- To improve overall user experience through a responsive and intuitive interface

1.4 Target Users

The system is designed to serve multiple types of users, each with specific roles and responsibilities:

- **Customers:** Browse products, place orders, make payments, and book maintenance appointments
 - **Technicians:** View assigned maintenance tasks and update their status
 - **Delivery:** Manage and track delivery of orders
 - **Managers:** Oversee inventory, pricing, and system operations
 - **Administrators:** Manage users, roles, and overall system configuration
-

1.5 Scope of the Project

This project focuses on developing a complete web-based solution that integrates e-commerce functionality with maintenance service management. It includes features such as user authentication, product management, order processing, payment handling, and appointment scheduling.

However, the system is limited to printer spare parts and related maintenance services. Future enhancements may include support for additional device categories, advanced analytics, and mobile application integration.

2. Requirements

2.1 Functional Requirements

2.1.1 Authentication & Authorization Management

- **User Registration:** Users can register by providing first name, last name, email, phone number, password, and address.
- **User Login:** Users authenticate using email and password and receive a JWT access token.
- **Logout:** Authenticated users can log out.
- **Role-Based Access Control:** System access is restricted based on assigned roles.

Role Hierarchy:

- **Admin :**

- Full system access.
- Can assign any role.
- Manages users, roles, payment methods, stock, services, suppliers, categories, and parts.

- **Manager:**

- Can view payments.
- Can assign roles below Manager level.
- Cannot manage payment methods or assign Admin role, Manages operational data (categories, suppliers, services, parts, prices, stock).

- **Technician**

- Can view maintenance appointments assigned to them.
- Can mark assigned maintenances as Completed once service is finished.

- **Delivery**

- Can view orders assigned to them for delivery.
- Can mark assigned orders as Completed once delivery is finished.

- **Customer (Any/Default user)**

- Can manage own account.
- Can manage own cart, orders, and maintenance appointments.
- Cannot access administrative modules.

2.1.2 User Management

- **Get User by ID:** Admin can retrieve any user information, User can retrieve own information only.
 - **Get All Users:** Retrieve all registered users (Admin only).
 - **Update User:** Admin can update any user, User can update own information only.
 - **Change Password:** Authenticated users can change their password.
 - **Delete User:** Remove user account; cascading deletes associated orders and maintenance records (Admin only).
 - **Automatic Cart Creation:** A cart is automatically created when a customer registers.
-

2.1.3 Role Management

- **Get All Roles:** Retrieve all system roles (Admin only).
 - **Get Role by ID:** Retrieve role details (Admin only).
 - **Assign Role to User:** Assign roles based on hierarchy (Normal users cannot assign roles).
 - **Remove Role from User:** Remove a role from a user based on hierarchy.
-

2.1.4 Supplier Management (Admin, Manager only)

- **Add Supplier:** Add new supplier with name, email, and phone.
- **Update Supplier:** Modify supplier details.
- **Delete Supplier:** Remove supplier; cascading deletes associated spare parts.

2.1.5 Category Management

- **Add Category:** Add new category with name and description (Admin, Manager only).
 - **Update Category:** Modify category details (Admin, Manager only).
 - **Delete Category:** Remove category; cascading deletes associated spare parts (Admin, Manager only).
 - **View Categories:** Retrieve all categories or a specific category by ID.
-

2.1.6 Service Management

- **Add Service:** Create service with name and price (Admin, Manager only).
 - **Update Service:** Modify service name or price (Admin, Manager only).
 - **Delete Service:** Remove service from system (Admin, Manager only).
 - **Get Service(s):** Retrieve all services or a specific service by ID.
 - **Service Price Snapshot:** Service price at time of order or maintenance creation is stored separately to preserve historical accuracy.
-

2.1.7 Spare Part Management

- **Add Spare Part:** Create spare part with name, description, category, supplier, and initial price (Admin, Manager only).
- **Update Spare Part:** Modify part details including category or supplier (Admin, Manager only).
- **Delete Spare Part:** Remove part; cascading deletes associated stock and price history (Admin, Manager only).
- **View Spare Parts:** Retrieve all parts or by ID.

2.1.8 Part Price Management (Price History)

- **Add Part Price:** Insert new price record when part price changes (Admin, Manager only).
 - **Get Part Prices:** Retrieve full price history of a part (Admin, Manager only).
 - **Get Latest Part Price:** Retrieve current active price of a part.
 - **Historical Preservation:** Previous price records remain stored for auditing and reporting purposes (Admin, Manager only).
-

2.1.9 Stock Management (Admin, Manager only)

- **Automatic Stock Creation:** Stock record is automatically created when a new spare part is added.
 - **Update Stock Value:** Set stock quantity directly.
 - **Adjust Stock Quantity:** Increment or decrement stock by a specific value.
 - **View Stock:** Retrieve all stock records or by part ID.
 - **Audit Tracking:** Stock records maintain Created_At and Updated_At timestamps.
-

2.1.10 Cart Management

- **Get Cart:** Retrieve authenticated user's cart and items.
- **Add Item to Cart:** Add spare part with quantity greater than zero.
- **Update Cart Item Quantity:** Modify quantity of an existing cart item.
- **Remove Cart Item:** Remove specific item from cart.
- **Clear Cart:** Remove all items from cart.
- **Cart-to-Order Conversion:** During checkout, cart items are automatically converted into order items.

2.1.11 Order Management

- **Create Order (Checkout):** Create order using cart contents and delivery address.
 - **Automatic Order Item Generation:** Order items are generated automatically from cart items.
 - **Order Status Management:** Orders can be marked as Pending or Completed.
 - **Complete Order:** Marks order as completed and sets completion timestamp.
 - **View Orders:** Retrieve user's orders or filter by status.
 - **Audit Tracking:** Orders maintain Created_At and Updated_At timestamps.
 - **Price Snapshot:** Unit price of each item is stored at order creation to prevent future price changes from affecting past orders.
-

2.1.12 Maintenance Management

- **Create Maintenance Appointment:** Customer schedules maintenance by providing address, description, and date/time.
- **Automatic Status Handling:** Maintenance is initially Scheduled.
- **Complete Maintenance:** Mark maintenance as completed and set Completed_At timestamp.
- **View Maintenances:** Retrieve user's maintenance records or filter by status (Scheduled, Completed).
- **Service Price Snapshot:** Service price at time of booking is stored to preserve historical accuracy.
- **Audit Tracking:** Maintenance records maintain Created_At and Updated_At timestamps.

2.1.13 Payment Management (Admin, Manager only)

- **View All Payments:** Retrieve all payment records.
 - **Get Payment by ID:** Retrieve specific payment details.
 - **Payment Methods Management:** Add, Update, Delete, and Retrieve Payment Method.
 - **Payment Status Tracking:** Payments support statuses such as Pending, Completed, and Failed.
 - **Exclusive Association Rule:** A payment is linked to either an order or a maintenance appointment, but not both.
-

2.2 Non-Functional Requirements

2.2.1 Performance Requirements

- The system shall support multiple concurrent users without significant performance degradation.
- API responses should generally be returned within **1–3 seconds** under normal operating conditions.
- The system must efficiently handle high-frequency operations such as cart updates, order creation, and maintenance scheduling.
- Database queries must be optimized to ensure fast retrieval of large datasets such as products, orders, and maintenance records.
- Pagination should be used for large data lists (e.g., orders, users, products) to improve performance.

2.2.2 Security Requirements

- The system must implement **JWT-based authentication** for secure login and session management.
 - Passwords must be securely hashed using strong hashing algorithms and must never be stored in plain text.
 - The system must enforce **Role-Based Access Control (RBAC)** to restrict unauthorized access to sensitive modules.
 - Sensitive operations (e.g., user management, stock updates, role assignment, payment records) must only be accessible to authorized roles.
 - All API endpoints must validate authentication tokens before processing requests.
 - The system must protect against common web vulnerabilities such as unauthorized access and data tampering.
-

2.2.3 Availability & Reliability

- The system should be designed to ensure high availability with minimal downtime.
- Critical operations such as order placement, maintenance booking, and stock updates must ensure data consistency.
- The system must handle runtime errors gracefully without affecting unrelated modules.
- Backend services should be designed in a modular way to ensure failure isolation between components.
- All transactional operations (orders, payments, maintenance completion) must maintain data integrity.

2.2.4 Scalability

- The system must support future growth in users, transactions, products, and maintenance requests.
 - The architecture (Spring Boot backend and React frontend) should allow horizontal scaling when deployed on cloud infrastructure.
 - The database schema should support extension without requiring major structural changes.
 - The system should support increased load through efficient API design and pagination mechanisms.
-

2.2.5 Usability

- The system must provide an intuitive and user-friendly interface for all roles (Customer, Admin, Manager, Technician, Delivery).
 - Each user role must have a dedicated dashboard tailored to their responsibilities.
 - Forms (registration, login, checkout, maintenance booking) must include validation and clear error messages.
 - The UI should be responsive and accessible across different screen sizes and devices.
 - Navigation should be simple and consistent across all modules.
-

2.2.6 Maintainability

- The system must follow a modular architecture separating frontend and backend components.
- The backend must follow a layered architecture (Controller, Service, Repository).
- Code must follow clean coding principles to ensure readability and maintainability.
- The system should allow easy modification of business logic such as pricing, stock management, and role permissions.
- Features should be loosely coupled to allow independent updates without affecting the entire system.

2.2.7 Data Integrity

- The system must ensure consistency between related entities such as orders, payments, stock, and maintenance records.
 - Price snapshots must be stored at the time of order or maintenance creation to preserve historical accuracy.
 - Stock levels must be updated consistently when orders are created or completed.
 - Deletion of entities (e.g., suppliers, categories, users) must handle cascading effects to avoid orphan records.
 - Transactions must be handled safely to prevent partial updates in critical operations.
-

2.2.8 Compatibility

- The system must be compatible with modern web browsers such as Chrome, Firefox, and Edge.
 - The frontend (React) must communicate seamlessly with the Spring Boot REST API.
 - The system should support deployment on standard web servers and cloud platforms.
 - The backend must expose RESTful APIs that can be consumed by any compatible frontend client.
-

2.2.9 Communication Requirements

- Communication between frontend and backend must be done using RESTful APIs with JSON format.
- All API responses must follow a consistent structure for ease of integration.
- Proper HTTP status codes (200, 201, 400, 401, 403, 404, 500) must be used for success and error handling.
- Requests must include proper validation to ensure data consistency and security.

5. Feasibility Study

5.1 Technical Feasibility

- The system is technically feasible as it is built using widely adopted and well-supported technologies:
 - The backend is developed using Spring Boot, which provides a robust framework for building scalable RESTful APIs.
 - The frontend is developed using ReactJS, ensuring a responsive and dynamic user interface.
 - The database is implemented using PostgreSQL, which supports relational data integrity and complex queries.
 - JWT authentication and Role-Based Access Control (RBAC) are used to ensure secure access management.
 - The chosen technologies are modern, stable, and well-documented, making the system highly feasible to implement and maintain.
-

5.2 Operational Feasibility

- The system is operationally feasible as it directly supports real-world business processes such as:
 - Online purchasing of spare parts
 - Maintenance appointment booking
 - Order delivery management
 - Inventory and stock tracking
 - Role-based system administration
 - Each user role (Customer, Admin, Manager, Technician, Delivery) has a dedicated dashboard, ensuring ease of use and clarity in operations.
- The system is designed to improve efficiency by automating manual tasks such as order tracking, stock updates, and maintenance scheduling.

5.3 Economic Feasibility

- The system is economically feasible due to the use of open-source technologies:
 - Spring Boot
 - ReactJS
 - PostgreSQL
 - Node/npm ecosystem libraries
 - There are no licensing costs associated with the core technologies used. Development and deployment costs are minimal and mainly depend on hosting infrastructure.
 - Additionally, the system reduces operational costs for businesses by automating order processing, inventory management, and maintenance scheduling.
-

3. Software Development Life Cycle (SDLC) – Agile Model

3.1 Overview of Agile Model

The Agile Software Development Model is an iterative and incremental approach to software development that focuses on flexibility, collaboration, and continuous improvement. Unlike traditional linear models, Agile divides the development process into smaller units called *iterations* or *sprints*, where each sprint delivers a functional part of the system.

Agile emphasizes close collaboration between team members and stakeholders, allowing requirements to evolve over time. Continuous feedback is incorporated throughout the development lifecycle, ensuring that the final product meets user needs and expectations.

3.2 Why Agile Model Was Chosen

The Agile model was selected for this project due to the dynamic and multi-functional nature of the system. The application includes multiple user roles such as customers, technicians, delivery personnel, managers, and administrators, each with distinct functionalities and interactions.

Additionally, the project involves several complex modules, including product management, order processing, payment handling, and maintenance appointment

scheduling. These features required flexibility in development, as requirements could evolve during implementation.

Agile allows the team to:

- Develop features incrementally
- Adapt to changing requirements
- Continuously test and improve the system
- Deliver working components at each stage

This makes Agile the most suitable approach for ensuring both efficiency and high quality.

3.3 Advantages of Agile Model

The Agile model provides several advantages in the development of this project:

- **Flexibility:** Requirements can be modified at any stage of development.
 - **Continuous Delivery:** Functional components are delivered in each sprint.
 - **Improved Collaboration:** Team members work closely and communicate regularly.
 - **Early Testing:** Issues are identified and resolved early in the process.
 - **Customer Satisfaction:** Frequent updates ensure the system aligns with user expectations.
-

3.4 Disadvantages of Agile Model

Despite its advantages, Agile also has some limitations:

- **Less Predictability:** It is difficult to estimate the total time and cost accurately.
- **Requires Strong Team Coordination:** Frequent communication is essential for success.
- **Scope Creep Risk:** Continuous changes may lead to uncontrolled expansion of features.
- **Documentation May Be Limited:** Agile focuses more on development than extensive documentation.

3.5 Project Sprints Plan

The project was developed using a series of structured sprints, where each sprint focused on delivering specific functionalities of the system.

Sprint	Description	Key Features
Sprint 1	System Integration & Role Management	Role-based access control, authentication, frontend-backend integration
Sprint 2	Maintenance Appointment System	Appointment booking, technician assignment, status tracking
Sprint 3	User Interface & Experience	UI improvements, form validation, error handling
Sprint 4	Advanced Features	Search & filtering, inventory alerts, dashboard analytics
Sprint 5	Testing & Deployment	System testing, security implementation, deployment, documentation

Each sprint followed the Agile cycle of planning, development, testing, and review. This ensured continuous improvement and delivery of a reliable and efficient system.

3. Use Case Specifications

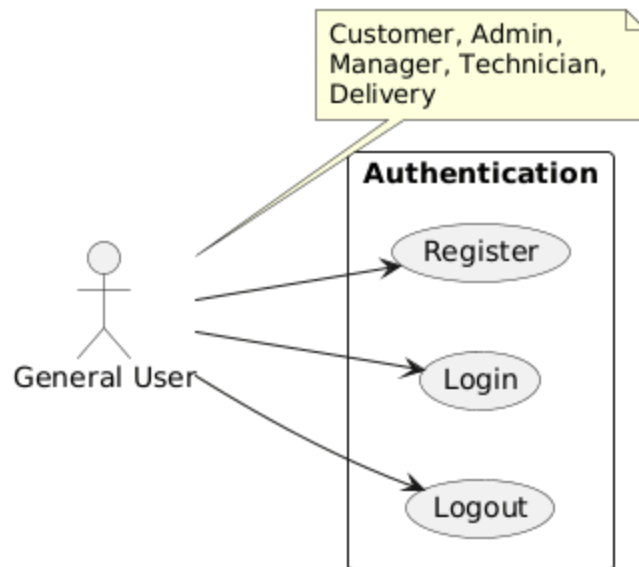
This section presents detailed descriptions of the system use cases. Each use case defines a specific interaction between an actor and the system, including the flow of events, preconditions, and outcomes.

3.1 Use Case Diagrams

3.1.1 Authentication Use Case Diagram

This diagram represents user authentication processes including registration, login, and logout.

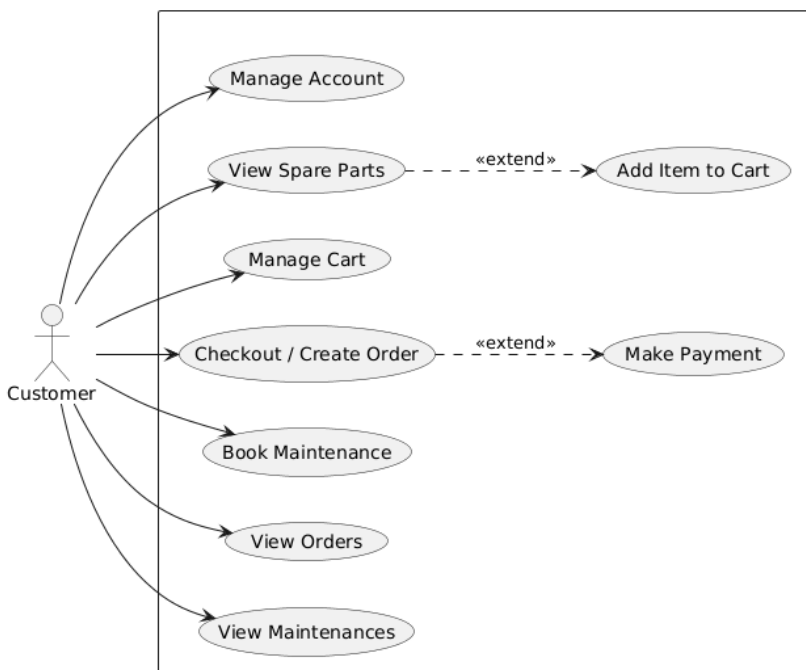
All system users interact with these functionalities to access the system securely.



3.1.2 Customer Use Case Diagram

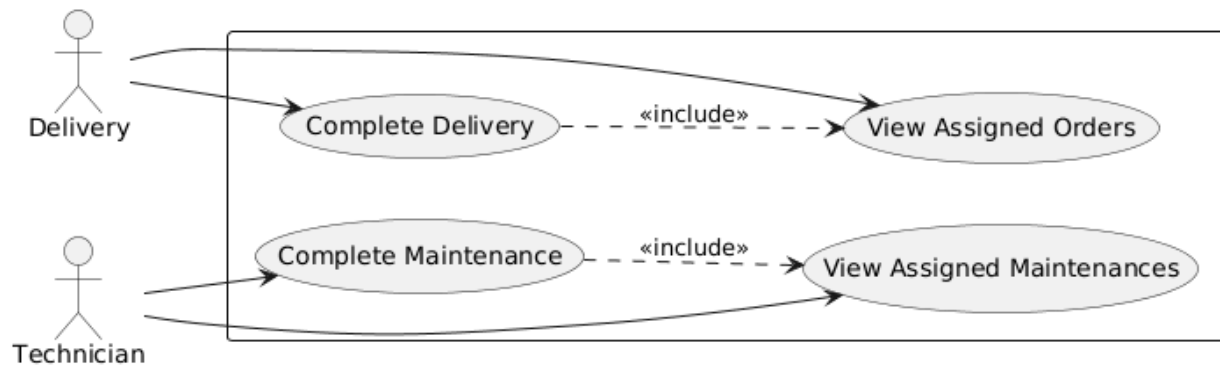
This diagram illustrates the interactions between the customer and the system.

Customers can browse spare parts, manage their cart, place orders, make payments, and schedule maintenance appointments.



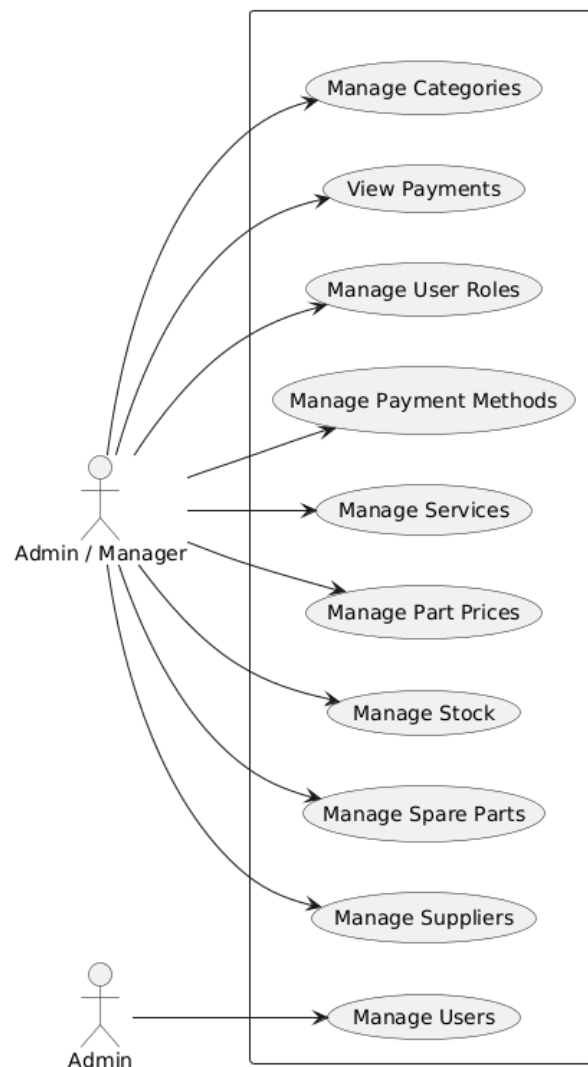
3.1.3 Delivery & Technician Diagram

This diagram represents operational roles responsible for executing services. Technicians handle maintenance appointments, while delivery personnel manage order deliveries.



3.1.4 Admin & Manager Use Case Diagram

This diagram shows system management functionalities performed by Admin and Manager roles. These include managing products, categories, suppliers, stock, payments, and users.



3.2 Use Cases Descriptions

3.2.1 Use Case: Register

ID	U-001
Name	Register
Description	Allows a new user to create an account by providing required personal information.
Actors	Customer (Any new user)
Preconditions	None
Postconditions	A new user account is created with the default role "Customer".
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. User opens registration page. 2. User enters required data. 3. System validates input. 4. System creates account. 5. System assigns default role. 6. System confirms registration.
Alternative Flow	3.1 Invalid data → error message displayed. 3.2 Email already exists → registration rejected.

3.2.2 Use Case: Login

ID	U-002
Name	Login
Description	Allows a registered user to authenticate using email and password.
Actors	All Users (Customer, Admin, Manager, Technician, Delivery)
Preconditions	User must be registered.
Postconditions	User is authenticated and redirected to their dashboard.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. User enters email and password. 2. System validates credentials. 3. System generates JWT token. 4. User is redirected to dashboard.
Alternative Flow	2.1 Invalid credentials → error message displayed.

3.2.3 Use Case: Logout

ID	U-003
Name	Logout
Description	Allows an authenticated user to log out securely from the system.
Actors	All Users
Preconditions	User must be logged in.
Postconditions	User session is terminated and token is invalidated.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. User selects logout option. 2. System invalidates session/token. 3. User is redirected to login page.
Alternative Flow	None

3.2.4 Use Case: View Spare Parts

ID	U-004
Name	View Spare Parts
Description	Allows customers to browse and view available printer spare parts with their details such as name, category, price, and availability.
Actors	Customer
Preconditions	None
Postconditions	Spare parts information is displayed to the user.
Include Use Case	None
Extend Use Case	Add Item to Cart
Base Use Case	None
Main Flow	1. User navigates to spare parts page. 2. System retrieves available spare parts. 3. System displays list of products with details. 4. User can view product details.
Alternative Flow	2.1 No products available → system displays empty catalog message.

3.2.5 Use Case: Manage Account

ID	U-005
Name	Manage Account
Description	Allows a user to view and update their personal information.
Actors	Customer
Preconditions	User must be logged in.
Postconditions	User information is updated successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. User navigates to account/profile page. 2. System displays user information. 3. User updates desired fields. 4. System validates input. 5. System saves updated information.
Alternative Flow	4.1 Invalid input → system rejects update.

3.2.6 Use Case: Add Item to Cart

ID	U-006
Name	Add Item to Cart
Description	Allows a customer to add a selected spare part to their cart.
Actors	Customer
Preconditions	User must be logged in. Product must exist.
Postconditions	Item is added to the cart successfully.
Include Use Case	View Spare Parts
Extend Use Case	None
Base Use Case	None
Main Flow	1. Customer selects product. 2. Customer enters quantity. 3. System validates quantity. 4. System adds item to cart.
Alternative Flow	3.1 Invalid quantity → request rejected.

3.2.7 Use Case: Manage Cart

ID	U-007
Name	Manage Cart
Description	Allows the customer to view and manage items in their cart, including updating quantities and removing items.
Actors	Customer
Preconditions	User must be logged in.
Postconditions	Cart is updated based on user actions.
Include Use Case	None.
Extend Use Case	None
Base Use Case	None
Main Flow	1. Customer opens cart page. 2. System displays all cart items. 3. Customer modifies item quantities or removes items. 4. System validates changes. 5. System updates the cart.
Alternative Flow	2.1 Cart is empty → system displays empty cart message. 3.1 Invalid quantity → system rejects update.

3.2.8 Use Case: Checkout (Create Order)

ID	U-008
Name	Checkout
Description	Allows a customer to place an order using items in their cart. The system automatically assigns a delivery personnel after order creation.
Actors	Customer
Preconditions	User must be logged in. Cart must not be empty.
Postconditions	Order is created and delivery personnel is automatically assigned.
Include Use Case	None
Extend Use Case	Make Payment
Base Use Case	None
Main Flow	1. Customer initiates checkout. 2. System retrieves cart items. 3. Customer confirms order details and selects payment method. 4. System creates order. 5. System generates order items. 6. System assigns delivery personnel. - If COD is selected → proceed without payment. - If Online Payment is selected → trigger Make Payment use case. 7. System clears cart.
Alternative Flow	2.1 Empty cart → checkout blocked.

3.2.9 Use Case: Make Payment

ID	U-009
Name	Make Payment
Description	Allows a customer to complete payment for an order using a selected payment method.
Actors	Customer
Preconditions	Customer must have a confirmed order or maintenance request awaiting payment.
Postconditions	Payment is processed and status is updated to “Completed” or “Failed”.
Include Use Case	None.
Extend Use Case	None
Base Use Case	Checkout
Main Flow	1. Customer proceeds to payment page. 2. System displays payment methods. 3. Customer selects payment method. 4. System processes payment request. 5. System updates payment status. 6. System confirms successful transaction.
Alternative Flow	4.1 Payment fails → system sets status to “Failed” and prompts retry. 4.2 Invalid payment method → system rejects request.

3.2.10 Use Case: Book Maintenance

ID	U-010
Name	Book Maintenance
Description	Allows a customer to schedule a maintenance service. The system automatically assigns an available technician.
Actors	Customer
Preconditions	User must be logged in.
Postconditions	Maintenance request is created and technician is automatically assigned.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Customer enters maintenance details. 2. System validates input. 3. System creates maintenance record. 4. System set status to “Scheduled”. 5. System automatically assigns technician. 6. System confirms appointment.
Alternative Flow	2.1 Invalid input → request rejected.

3.2.11 Use Case: View Orders

ID	U-011
Name	View Orders
Description	Allows users to view order history or manage system-wide orders depending on their role.
Actors	Customer
Preconditions	User must be logged in.
Postconditions	Order records are displayed successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. User navigates to account page. 2. System retrieves orders. 3. System displays order list. 4. User views order details if needed.
Alternative Flow	2.1 No orders found → system displays empty state message.

3.2.12 Use Case: View Maintenance

ID	U-012
Name	View Maintenance
Description	Allows users to view maintenance appointments based on their role and assignments.
Actors	Customer
Preconditions	User must be logged in.
Postconditions	Maintenance records are displayed successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. User navigates to account page. 2. System retrieves maintenances. 3. System displays maintenance list. 4. User views.
Alternative Flow	2.1 No records found → empty state displayed.

3.2.13 Use Case: View Assigned Orders

ID	U-013
Name	View Assigned Orders
Description	Allows delivery personnel to view orders assigned to them.
Actors	Delivery
Preconditions	User must be logged in with Delivery role.
Postconditions	Assigned orders are displayed.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Delivery personnel opens dashboard. 2. System retrieves assigned orders. 3. System displays list of orders. 4. User views order details if needed.
Alternative Flow	2.1 No assigned orders → system displays empty state.

3.2.14 Use Case: Complete Delivery

ID	U-014
Name	Complete Delivery
Description	Allows delivery personnel to mark an order as completed. If the order is Cash on Delivery (COD), the system automatically creates a payment record.
Actors	Delivery
Preconditions	Order must be assigned and in “Pending” status.
Postconditions	Order is marked as completed and payment is recorded if applicable.
Include Use Case	View Assigned Orders
Extend Use Case	None
Base Use Case	None
Main Flow	1. Delivery personnel selects an assigned order. 2. User marks order as “Completed”. 3. System updates order status. - If payment method is COD: a. System creates payment record. b. System sets payment status to “Completed”. 5. System confirms completion.
Alternative Flow	1.1 Order not assigned → action denied. 2.1 Order already completed → action rejected.

3.2.15 Use Case: View Assigned Maintenance

ID	U-015
Name	View Assigned Maintenance
Description	Allows technicians to view maintenance appointments assigned to them.
Actors	Technician
Preconditions	User must be logged in with Technician role.
Postconditions	Assigned maintenance records are displayed.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Technician opens dashboard. 2. System retrieves assigned maintenance tasks. 3. System displays maintenance list. 4. Technician views details if needed.
Alternative Flow	2.1 No assigned tasks → system displays empty state.

3.2.16 Use Case: Complete Maintenance

ID	U-016
Name	Complete Maintenance
Description	Allows a technician to mark a maintenance appointment as completed. The system automatically creates a payment record for the service.
Actors	Technician
Preconditions	Maintenance must be assigned and in “Scheduled” status.
Postconditions	Maintenance is marked as completed and payment is recorded.
Include Use Case	View Assigned Maintenance
Extend Use Case	None
Base Use Case	None
Main Flow	1. Technician selects assigned maintenance task. 2. Technician marks it as “Completed”. 3. System updates maintenance status. 4. System creates payment record. 5. System sets payment status to “Completed”. 6. System confirms completion.
Alternative Flow	1.1 Task not assigned → action denied. 2.1 Task already completed → action rejected. 2.2 Current date is before scheduled date → action rejected.

3.2.17 Use Case: Manage Categories

ID	U-017
Name	Manage Categories
Description	Allows Admin or Manager to manage product categories used to organize spare parts.
Actors	Admin, Manager
Preconditions	User must be authenticated with proper role.
Postconditions	Category data is created, updated, or deleted successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens categories dashboard. 2. System displays all categories. 3. Admin/Manager adds, updates, or deletes a category. 4. System updates database.
Alternative Flow	3.1 Invalid input → system rejects request. 3.2 Category in use → deletion blocked.

3.2.18 Use Case: Manage Suppliers

ID	U-018
Name	Manage Suppliers
Description	Allows Admin or Manager to manage supplier information for spare parts.
Actors	Admin, Manager
Preconditions	User must be authenticated with proper role.
Postconditions	Supplier data is stored or updated successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens suppliers dashboard. 2. System displays all suppliers. 3. Admin/Manager adds, edits, or deletes supplier. 4. System updates supplier data.
Alternative Flow	3.1 Invalid data → system rejects request. 3.2 Supplier linked to parts → deletion restricted (if applicable).

3.2.19 Use Case: Manage Spare Parts

ID	U-019
Name	Manage Spare Parts
Description	Allows Admin or Manager to add, update, view, and delete spare parts.
Actors	Admin, Manager
Preconditions	User must be logged in with proper role.
Postconditions	Product data is updated in the system.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens products dashboard. 2. System displays all spare parts. 3. Admin/Manager adds/edits/deletes product. 4. System updates database.
Alternative Flow	3.1 Invalid product data → system rejects request.

3.2.20 Use Case: Manage Stock

ID	U-020
Name	Manage Stock
Description	Allows Admin or Manager to update and track stock quantities of spare parts.
Actors	Admin, Manager
Preconditions	Spare part must exist.
Postconditions	Stock levels are updated.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager selects spare part from dashboard. 2. System displays stock details. 3. Admin/Manager updates quantity. 4. System saves updated stock.
Alternative Flow	3.1 Invalid quantity → system rejects update.

3.2.21 Use Case: Manage Part Prices

ID	U-021
Name	Manage Part Prices
Description	Allows Admin and Manager to manage spare part pricing by adding new price records and viewing price history.
Actors	Admin, Manager
Preconditions	Spare part must exist.
Postconditions	Price records are updated and historical data is preserved.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager selects a spare part from dashboard. 2. System displays current price and price history. 3. Admin/Manager adds a new price. 4. System creates a new price record. 5. System marks it as the latest price.
Alternative Flow	3.1 Invalid price → system rejects request.

3.2.22 Use Case: Manage Services

ID	U-022
Name	Manage Services
Description	Allows Admin and Manager to manage maintenance services, including creating, updating, viewing, and deleting service definitions and their associated prices.
Actors	Admin, Manager
Preconditions	User must be authenticated with appropriate role.
Postconditions	Service data is updated successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens services management dashboard. 2. System displays all services. 3. Admin/Manager adds, updates, or deletes a service. 4. System validates input. 5. System updates service data.
Alternative Flow	3.1 Invalid data → system rejects request.

3.2.23 Use Case: Manage Payment Methods

ID	U-023
Name	Manage Payment Methods
Description	Allows Admin and Manager to manage available payment methods used in the system.
Actors	Admin, Manager
Preconditions	User must be authenticated with appropriate role.
Postconditions	Payment method data is updated successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens payment methods section. 2. System displays available payment methods. 3. Admin/Manager adds, updates, or deletes a payment method. 4. System validates changes. 5. System updates database.
Alternative Flow	3.1 Invalid data → system rejects request.

3.2.24 Use Case: Manage User Roles

ID	U-024
Name	Manage User Roles
Description	Allows Admin and Manager to view system roles, view user roles, and assign or remove roles with restrictions based on hierarchy rules.
Actors	Admin, Manager
Preconditions	User must be authenticated with appropriate role.
Postconditions	Payment method data is updated successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens role management dashboard. 2. System displays available roles and users. 3. Admin/Manager selects a target user. 4. Admin/Manager assigns or removes roles. 5. System validates role hierarchy rules. 6. System updates user roles.
Alternative Flow	5.1 Invalid role assignment → system rejects request.

Business Rules:

- Admin can assign any role but cannot remove their own Admin role.
- Manager can only assign Customer, Technician, or Delivery roles.
- Manager cannot assign Admin or Manager roles.
- Manager cannot remove their own Manager role.

3.2.25 Use Case: View Payments

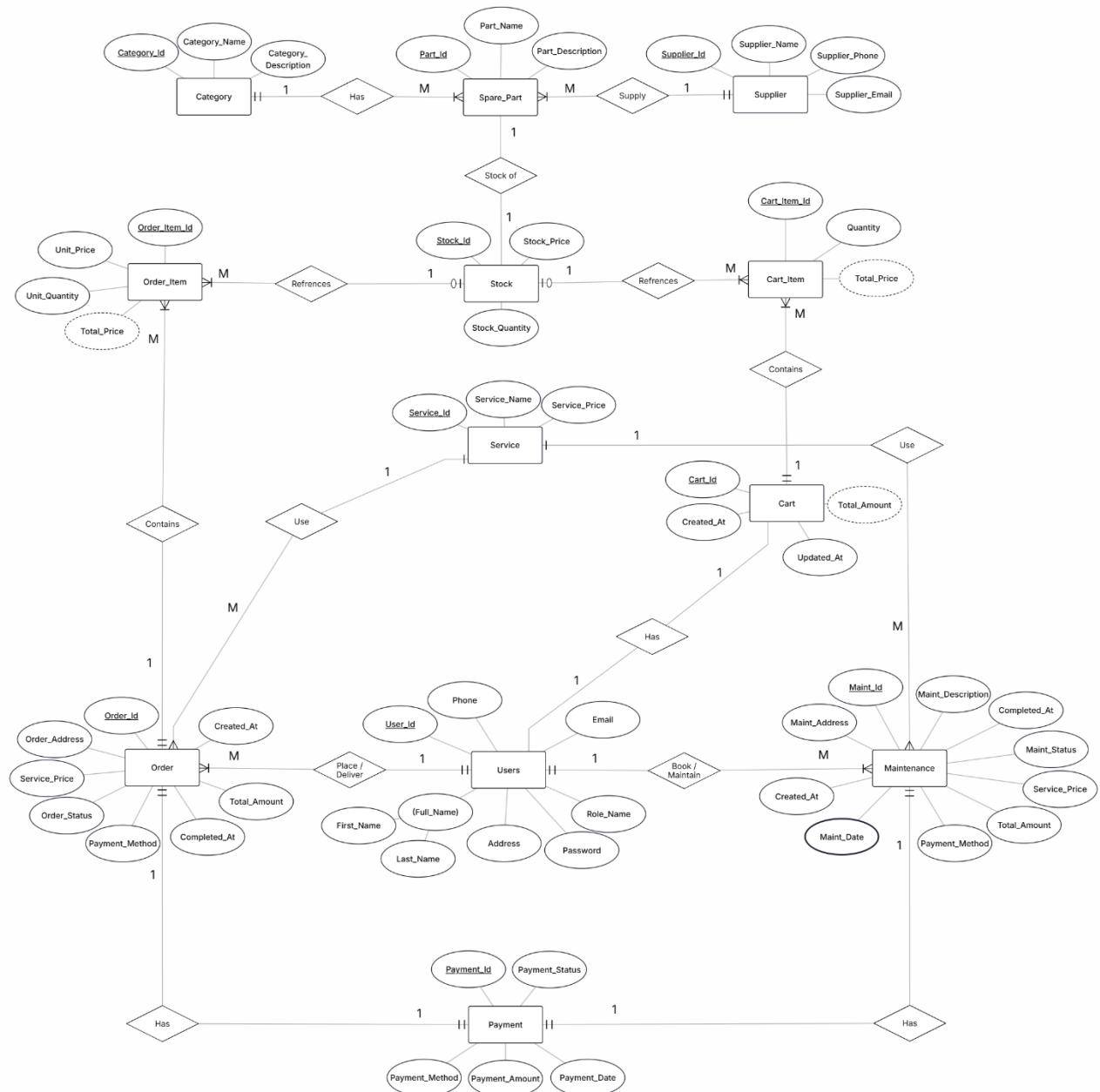
ID	U-025
Name	View Payments
Description	Allows Admin or Manager to view and retrieve payment records.
Actors	Admin, Manager
Preconditions	Payment records must exist.
Postconditions	Payment data is retrieved.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin/Manager opens payment dashboard. 2. System retrieves payment records. 3. Admin/Manager views payment details. 4. System displays status information.
Alternative Flow	2.1 No records → empty state displayed.

3.2.26 Use Case: Manage Users

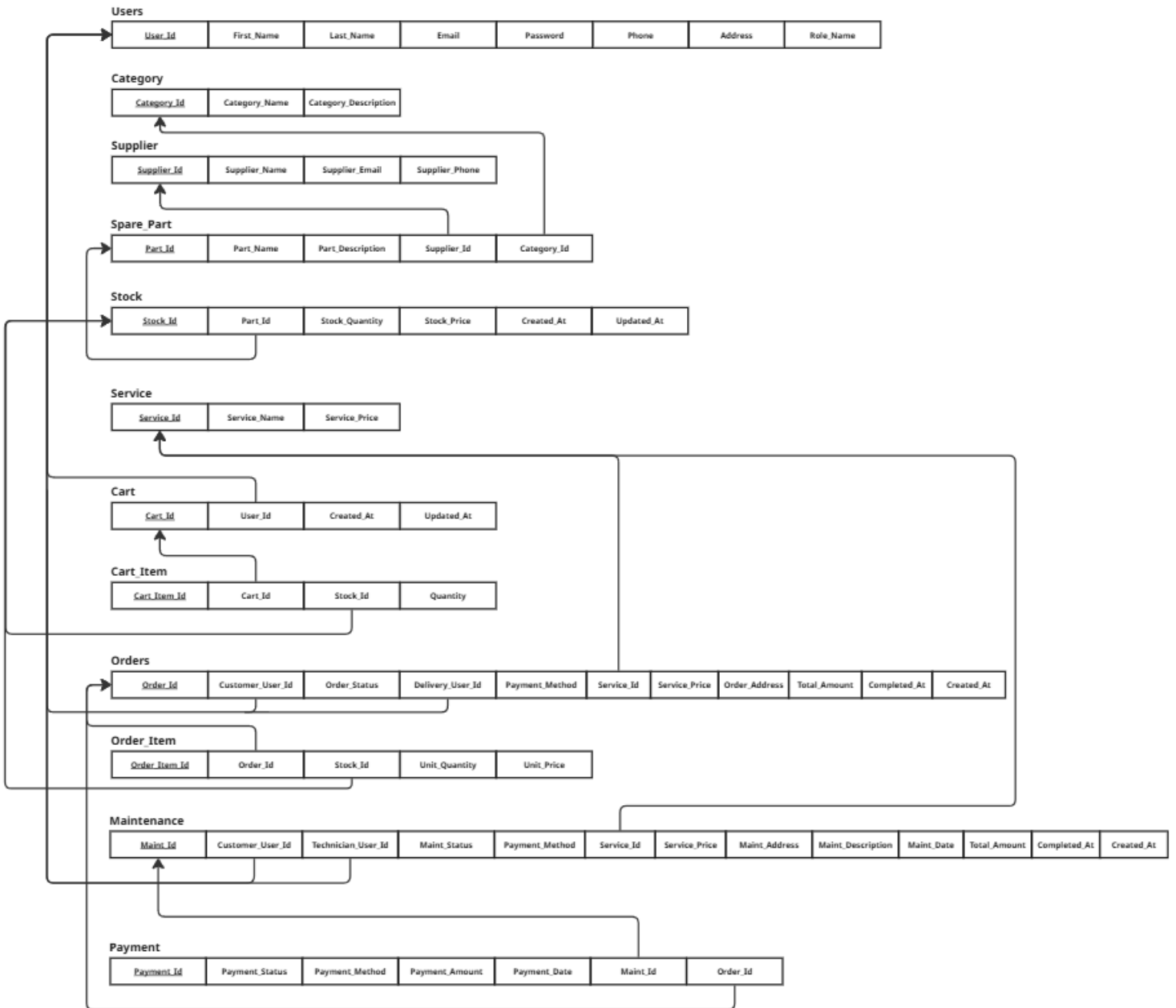
ID	U-026
Name	Manage Users
Description	Allows Admin to view, update, and delete user accounts in the system.
Actors	Admin
Preconditions	Admin must be logged in.
Postconditions	User data is updated or removed successfully.
Include Use Case	None
Extend Use Case	None
Base Use Case	None
Main Flow	1. Admin opens user management dashboard. 2. System displays all users. 3. Admin selects a user. 4. Admin updates or deletes user. 5. System applies changes.
Alternative Flow	4.1 Invalid operation → system rejects request.

5. Database Design

5.1 Entity Relationship Diagram (ERD)

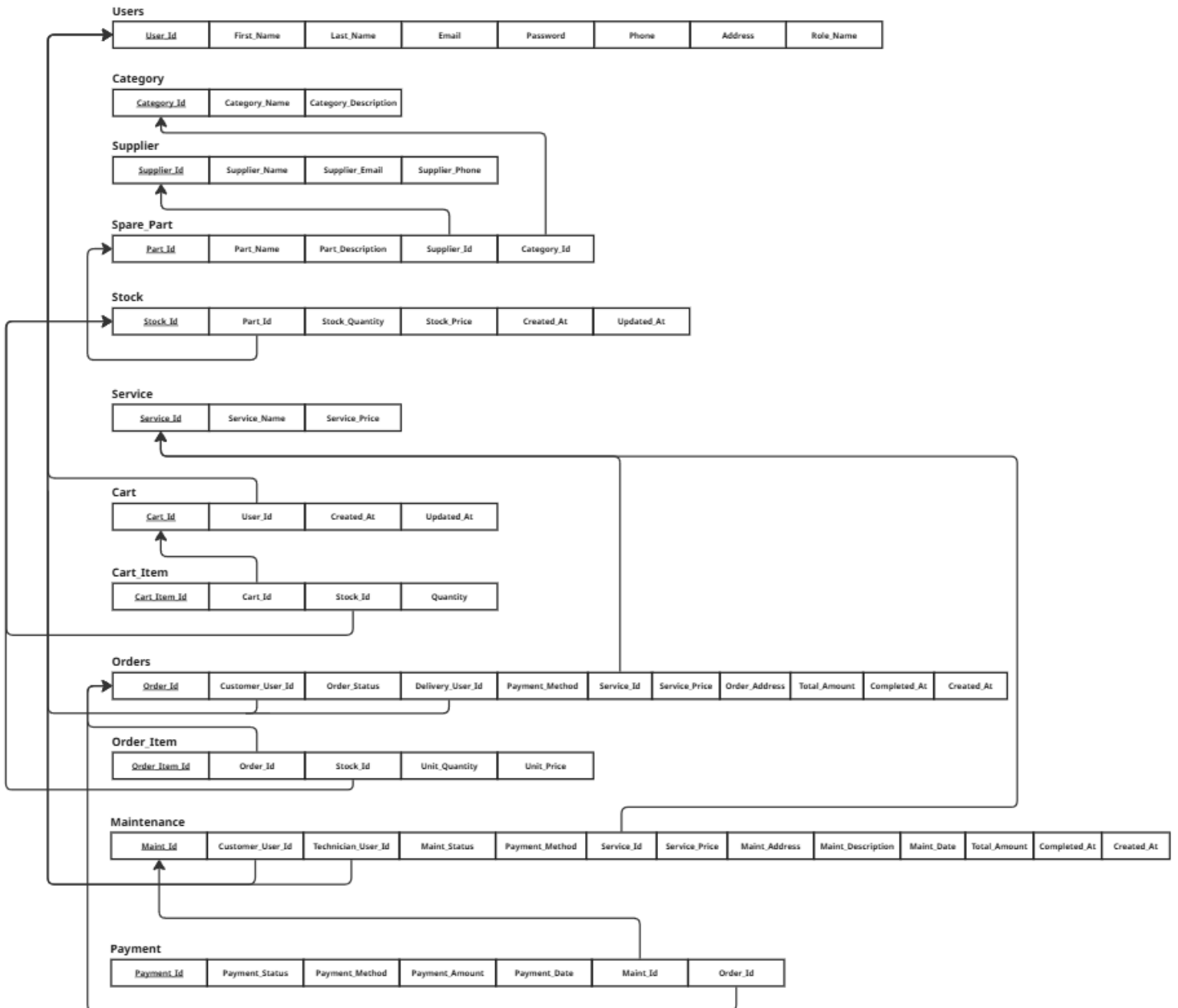


5.2 Mapping

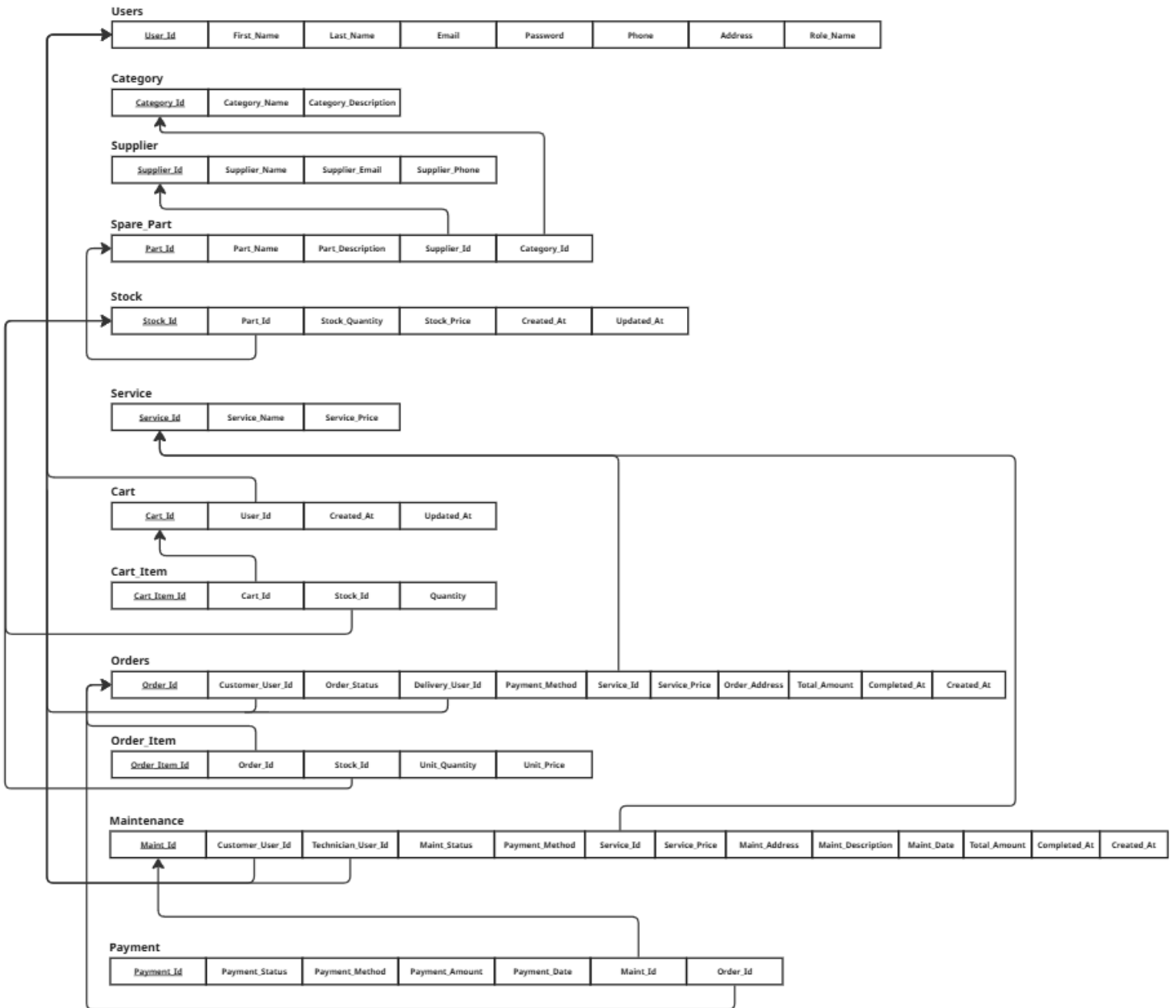


5.3 Normalization:

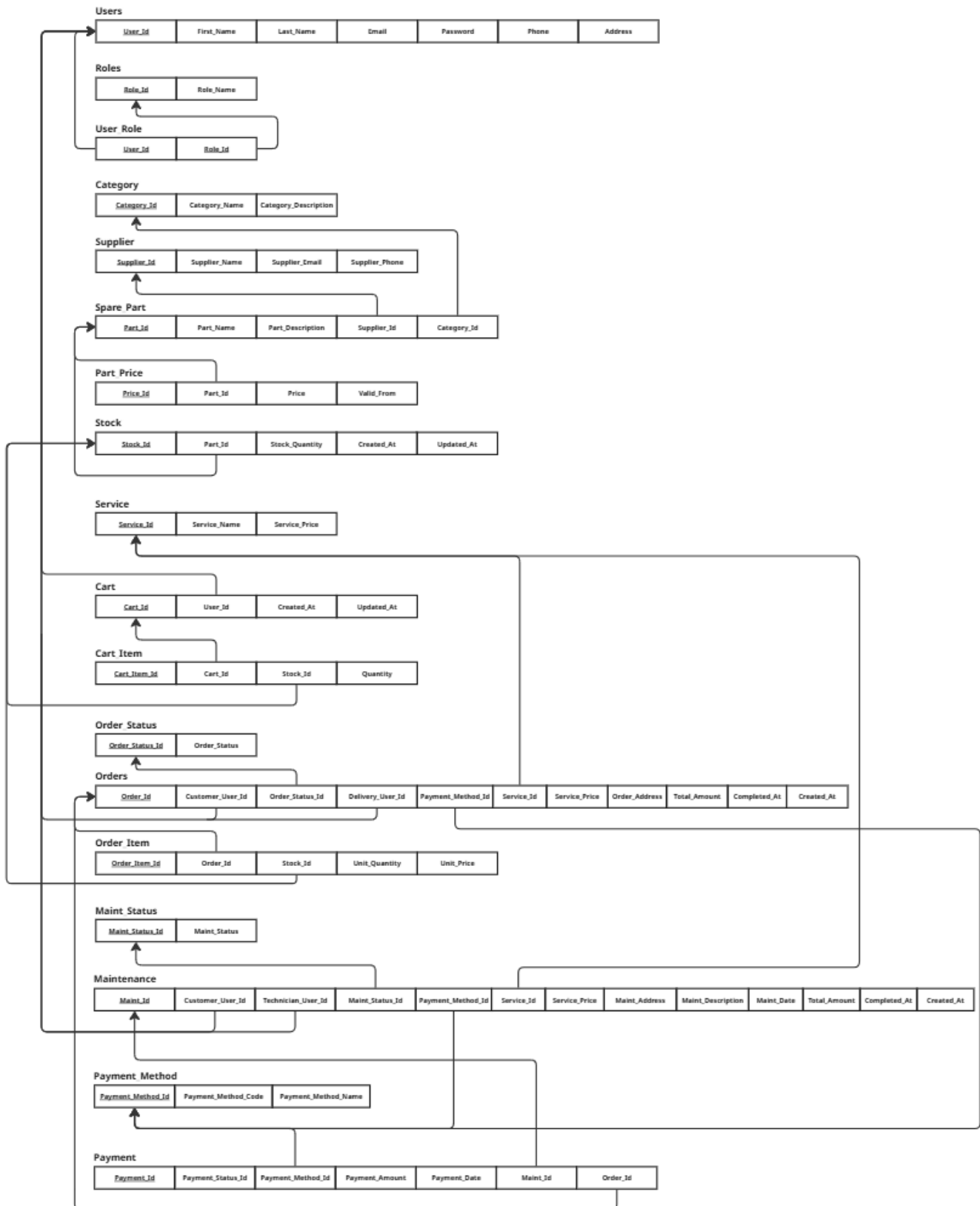
5.3.1 First Normal Form (1NF)



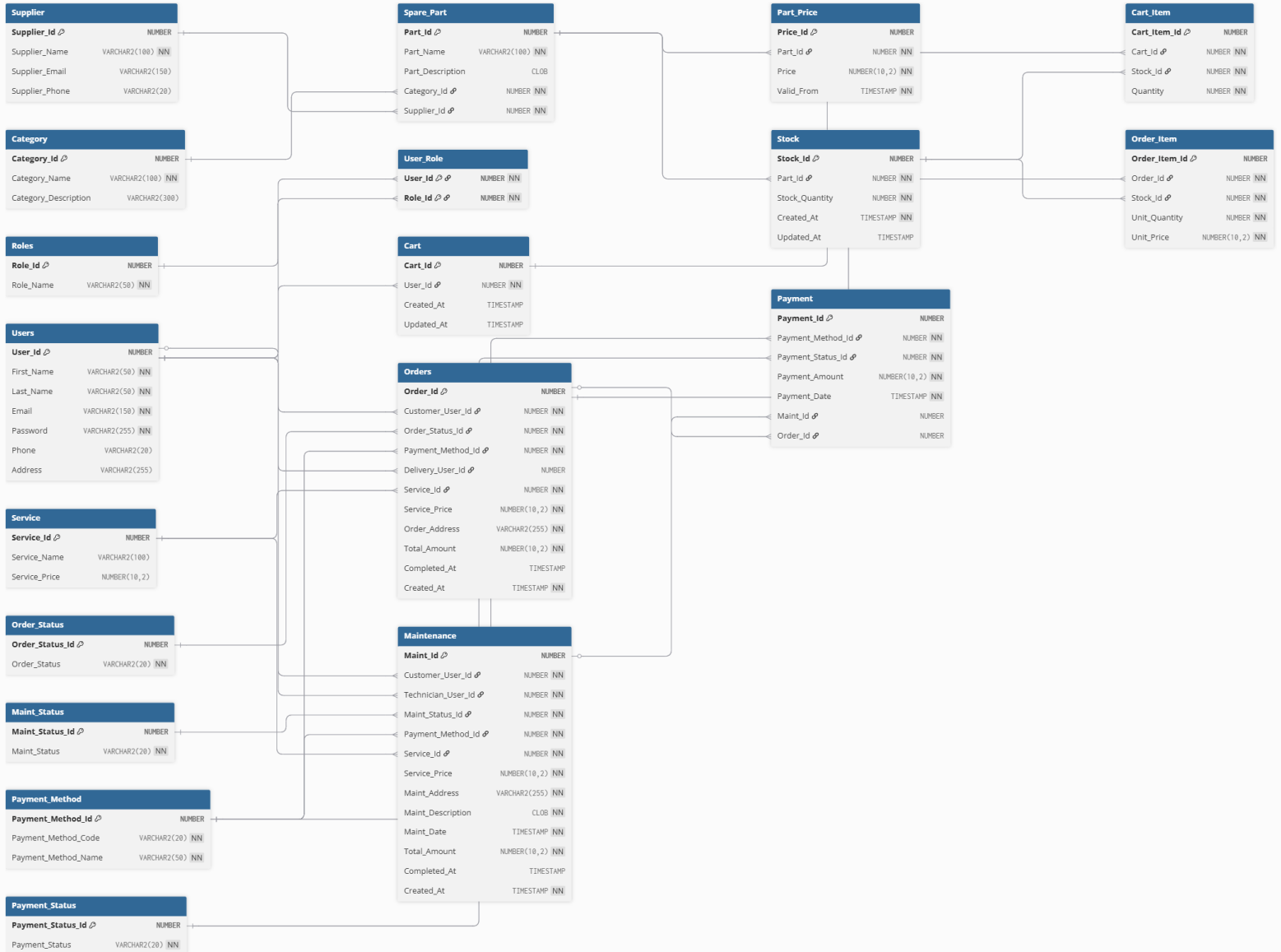
5.3.2 Second Normal Form (2NF)

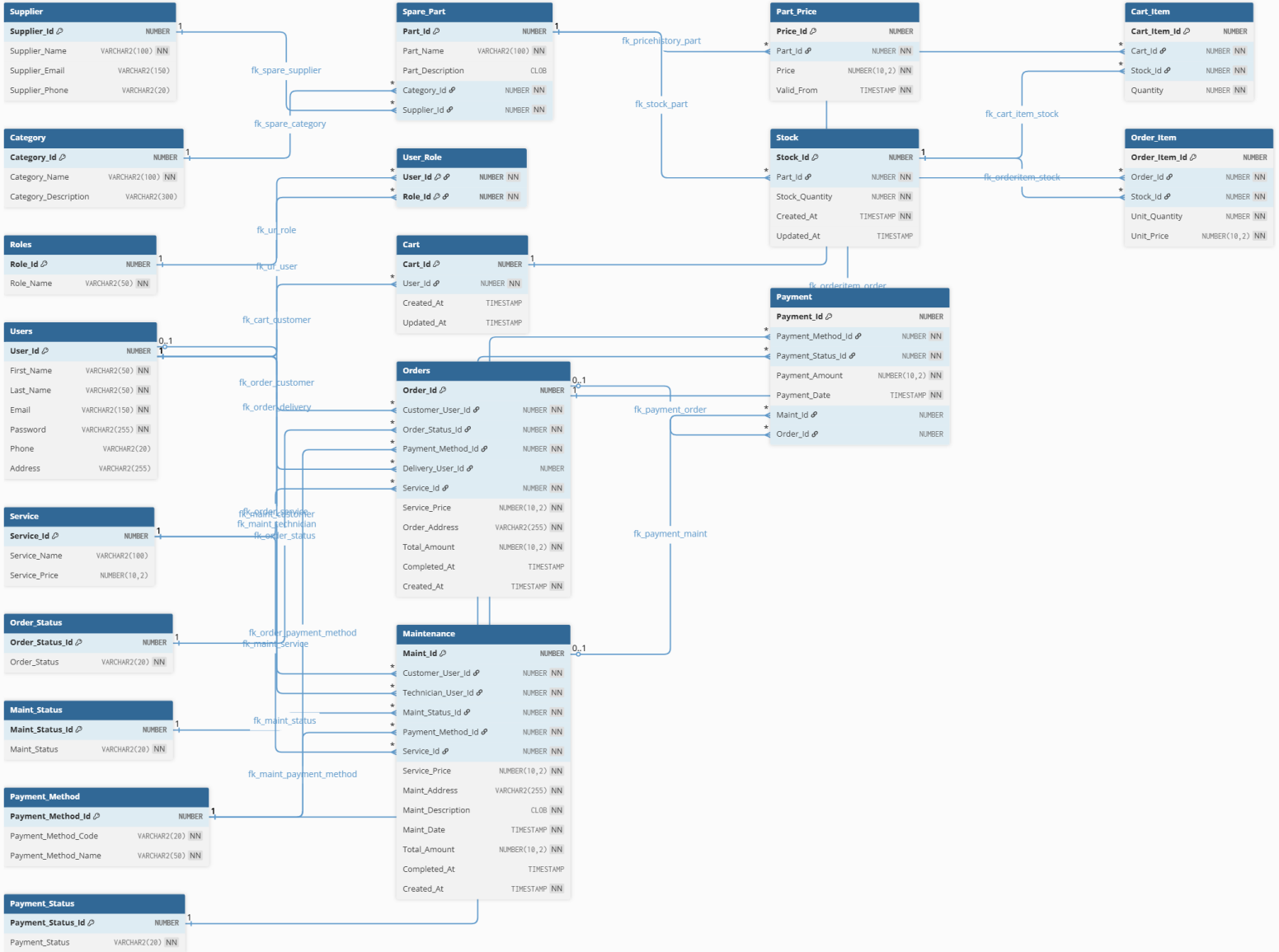


5.3.3 Third Normal Form (3NF)



5.4 Schema

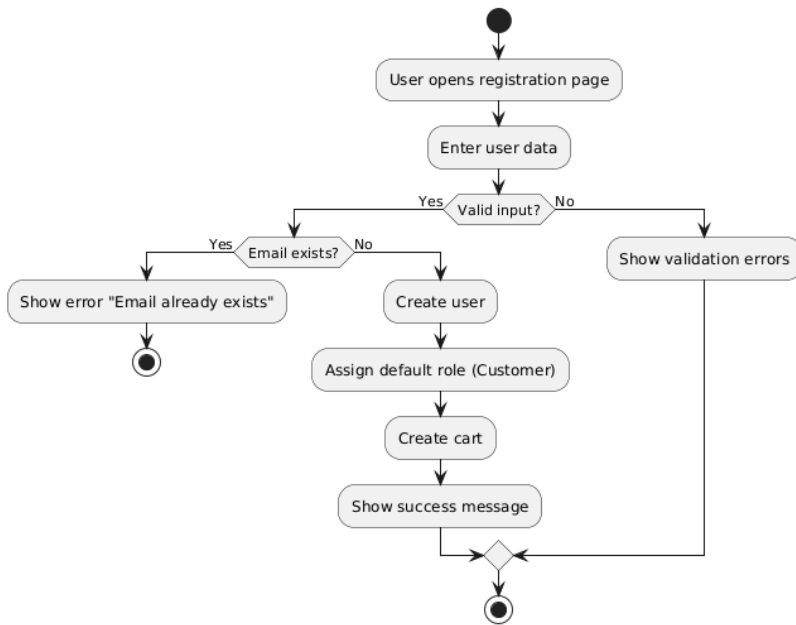




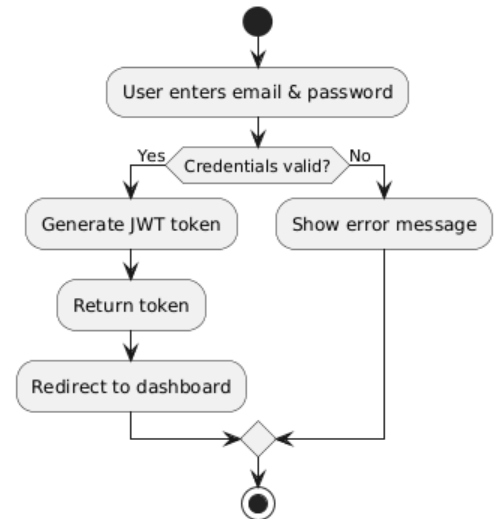
6. Diagrams:

6.1 Activity Diagrams:

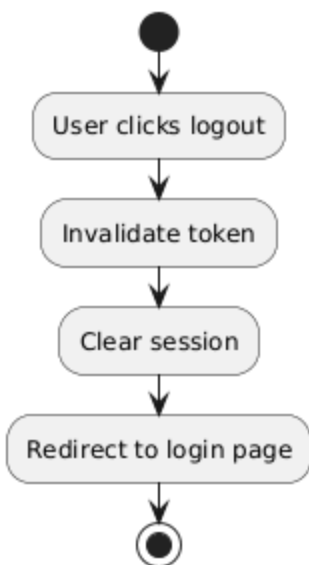
Register (U-001)



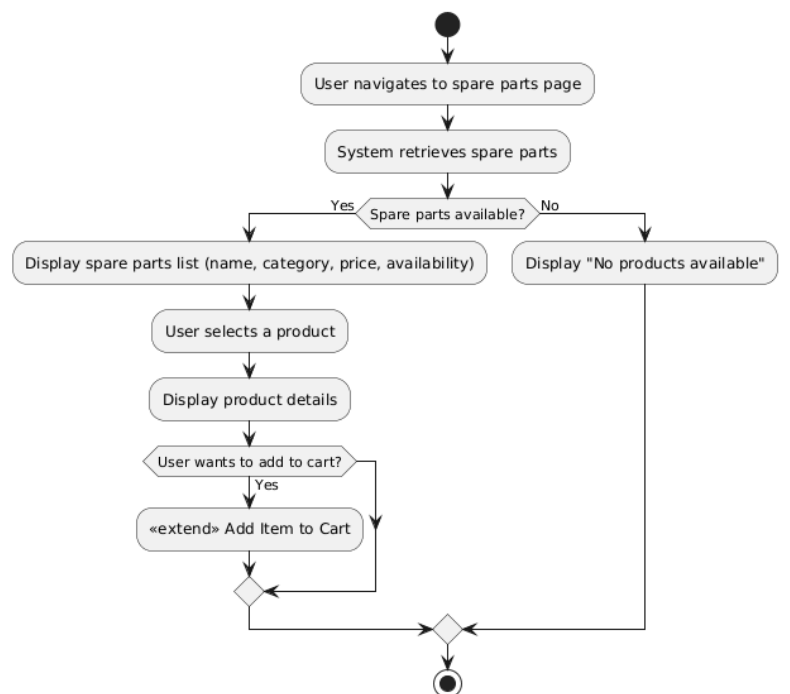
Login (U-002)



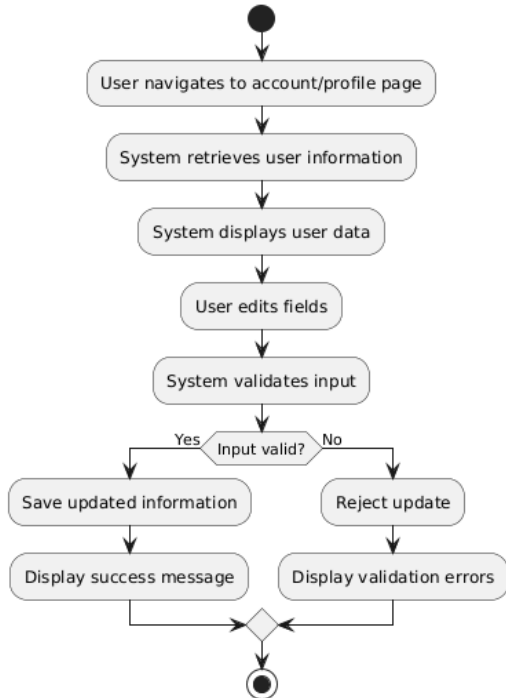
Logout (U-003)



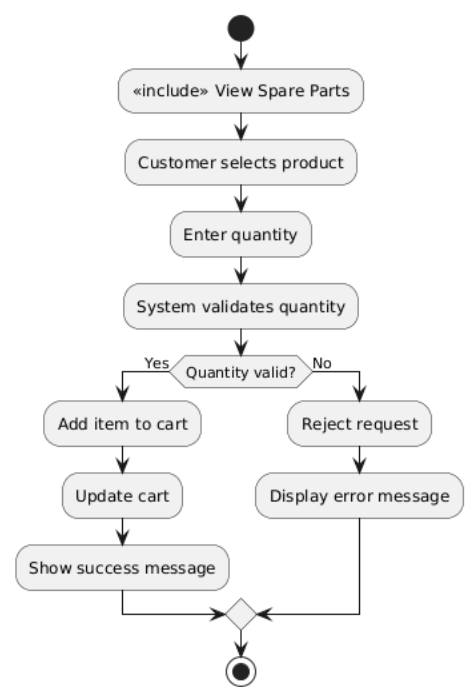
View Spare Parts (U-004)



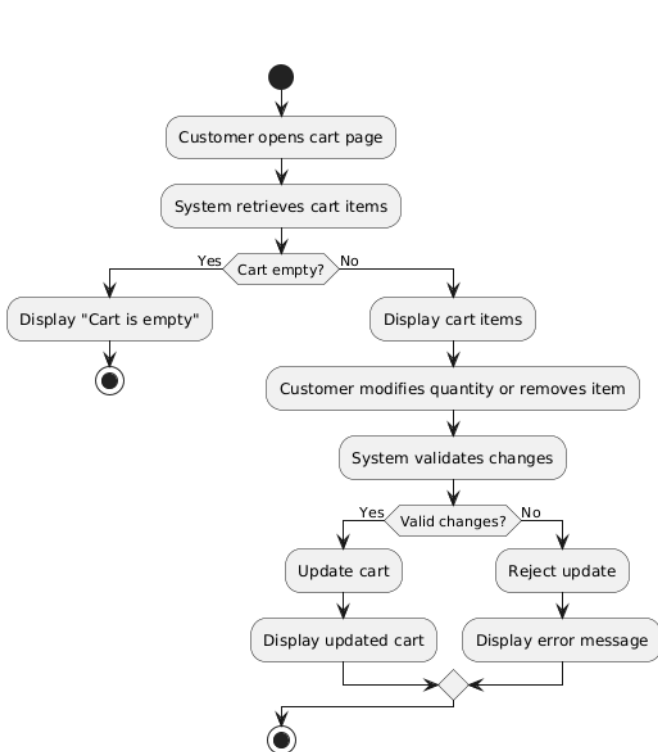
Manage Account (U-005)



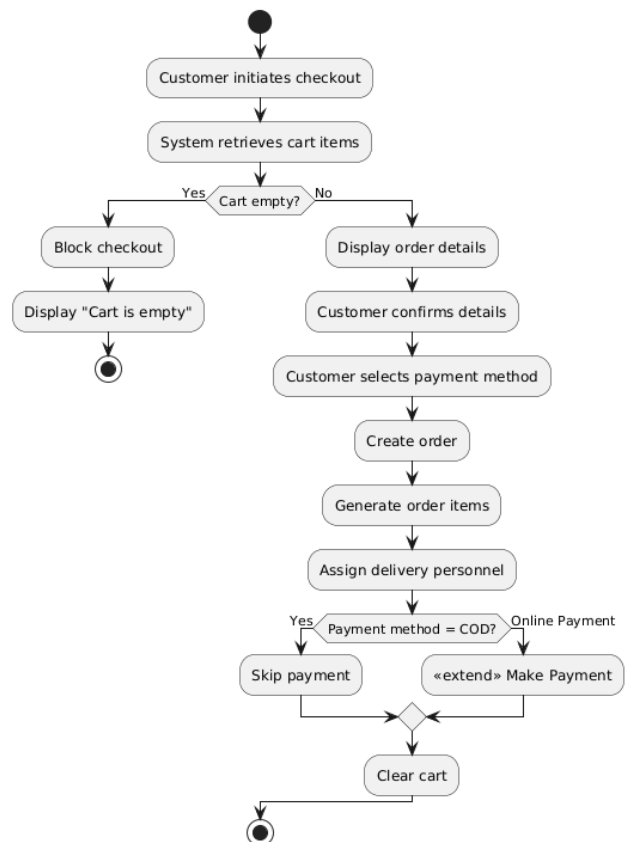
Add Item to Cart (U-006)



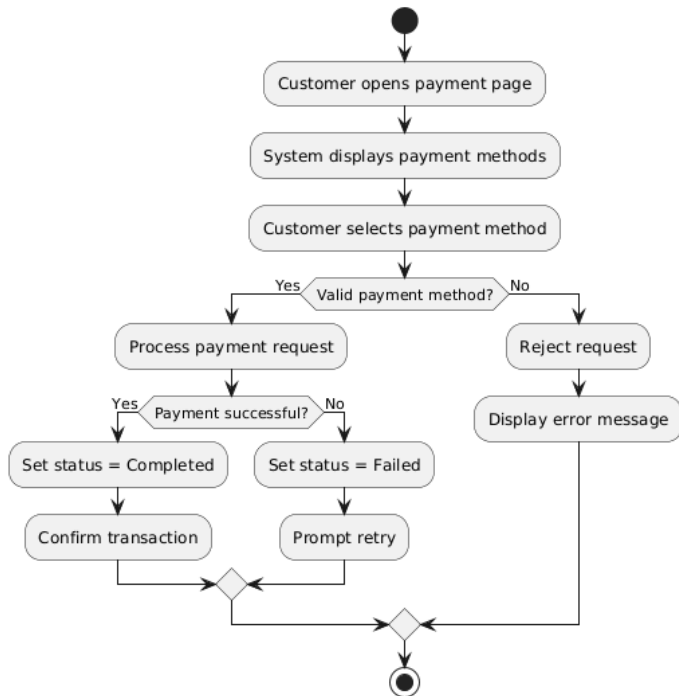
Manage Cart (U-007)



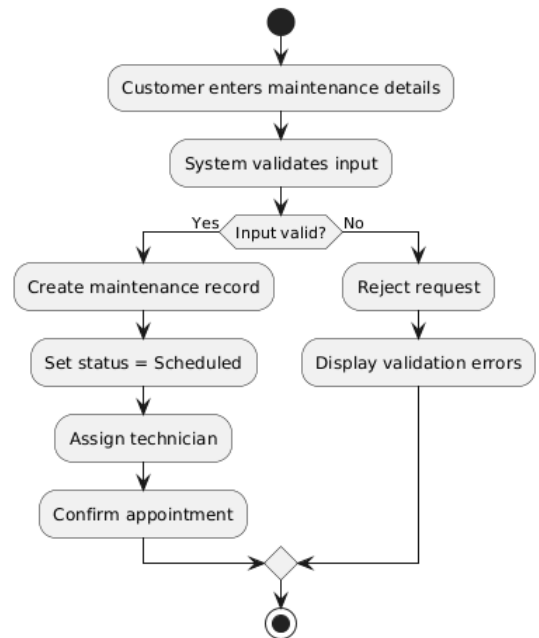
Checkout (U-008)



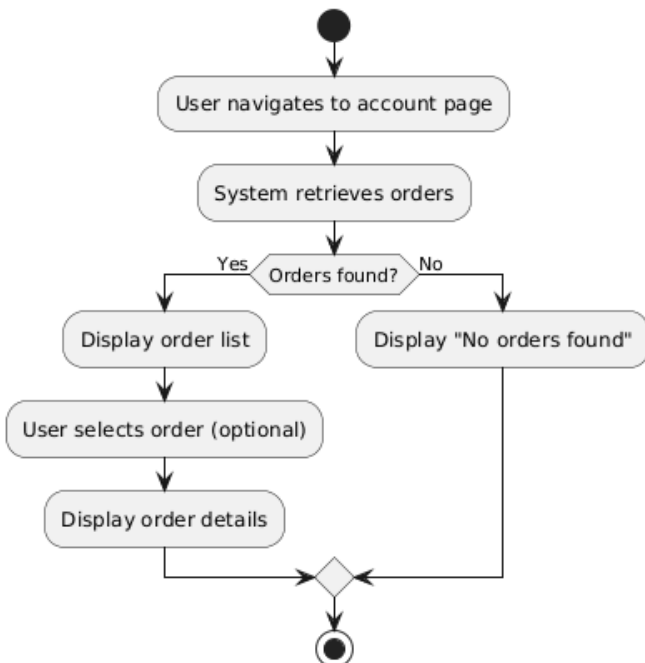
Make Payment (U-009)



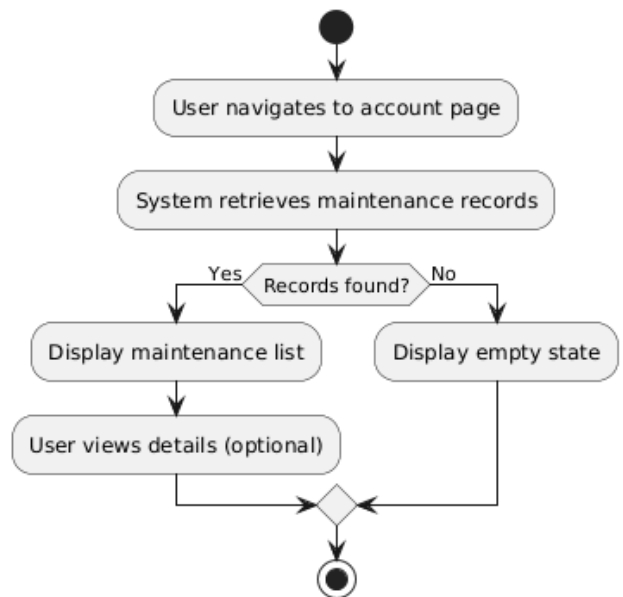
Book Maintenance (U-010)



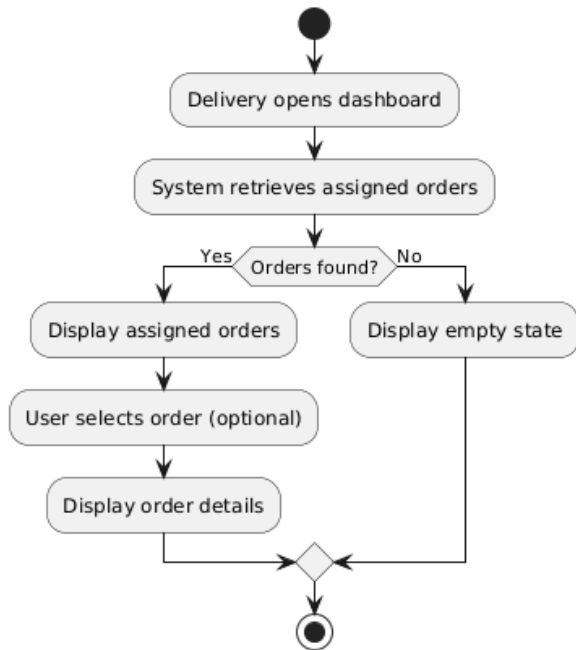
View Orders (U-011)



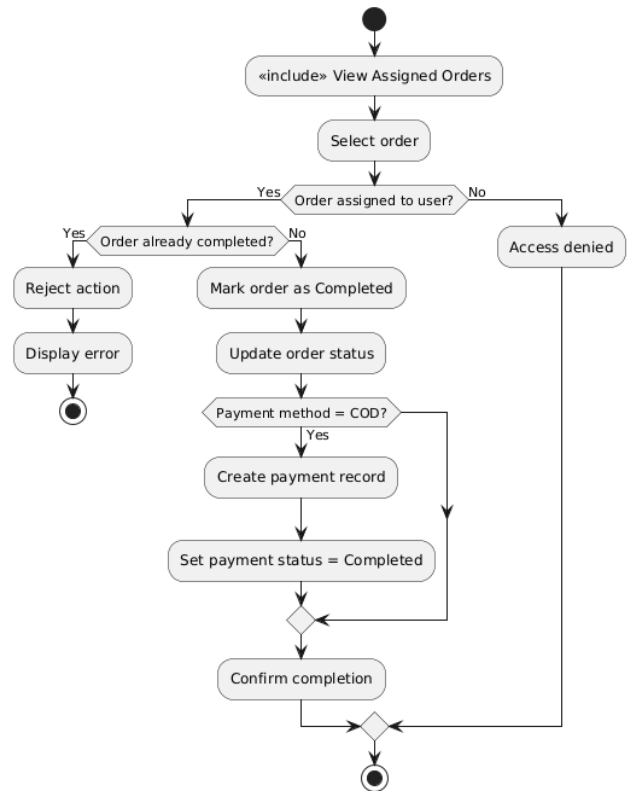
View Maintenance (U-012)



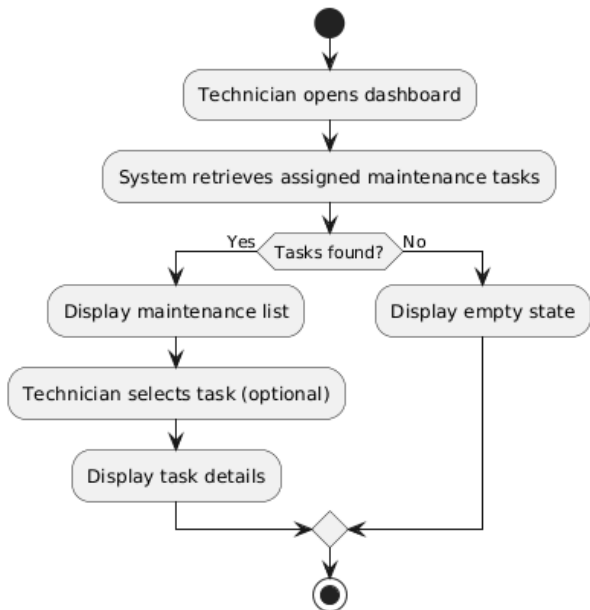
View Assigned Orders (U-013)



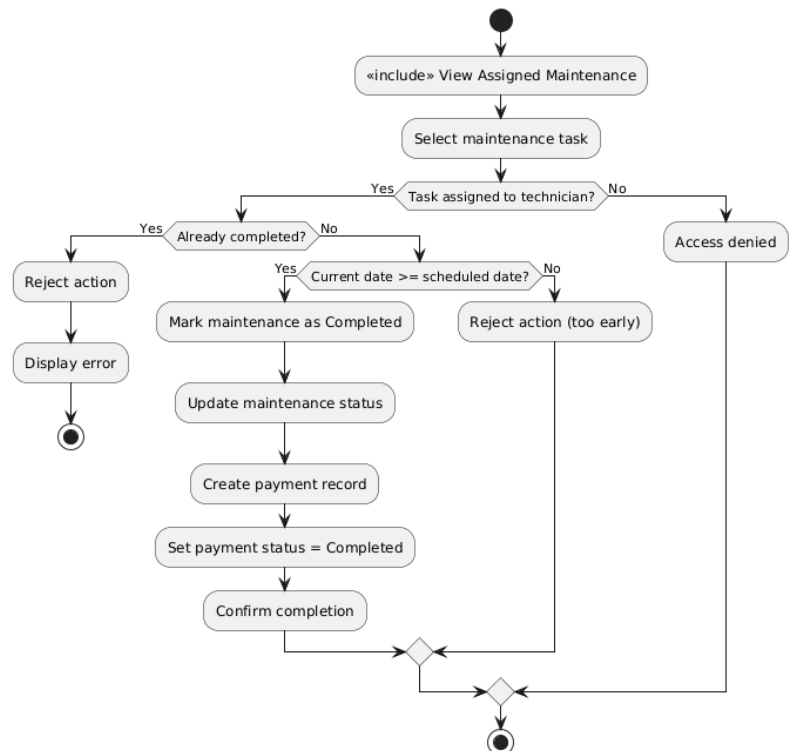
Complete Delivery (U-014)



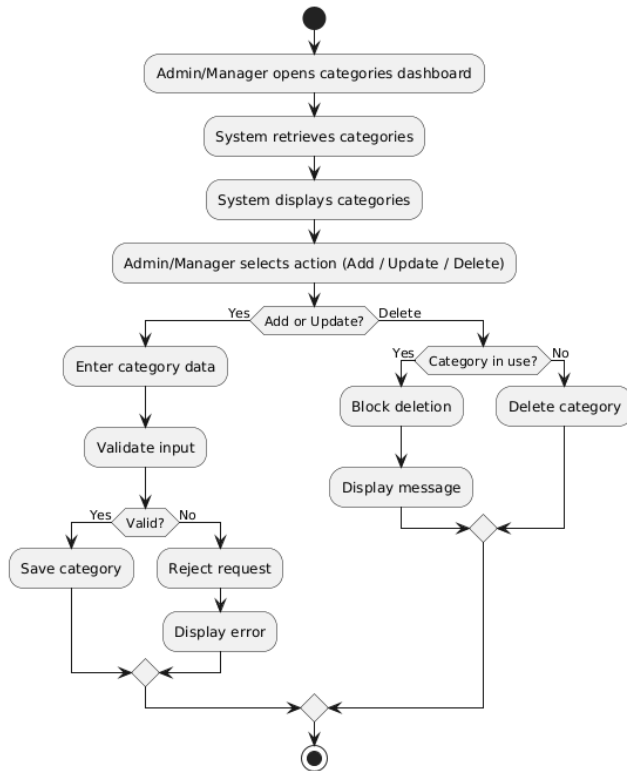
View Assigned Maintenance (U-015)



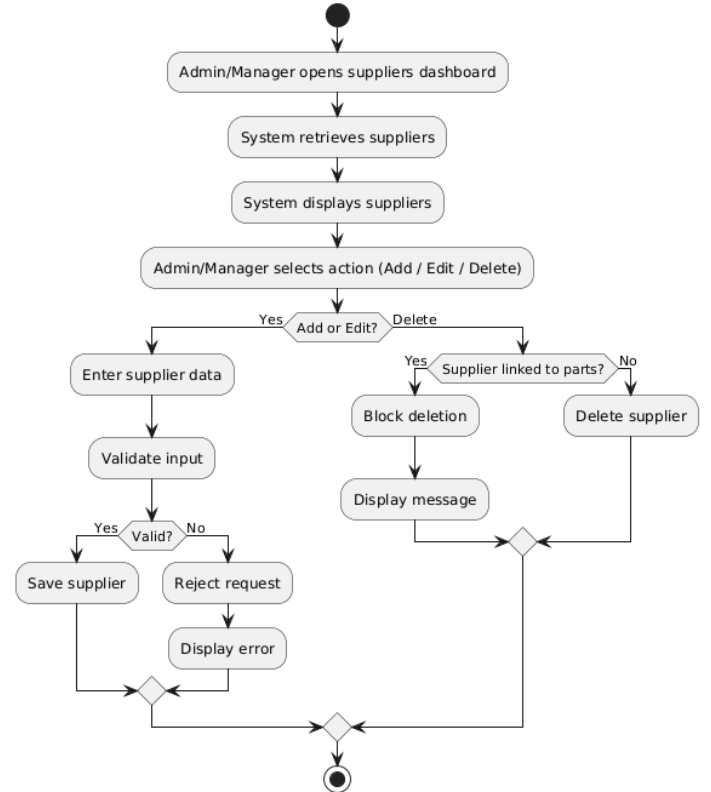
Complete Maintenance (U-016)



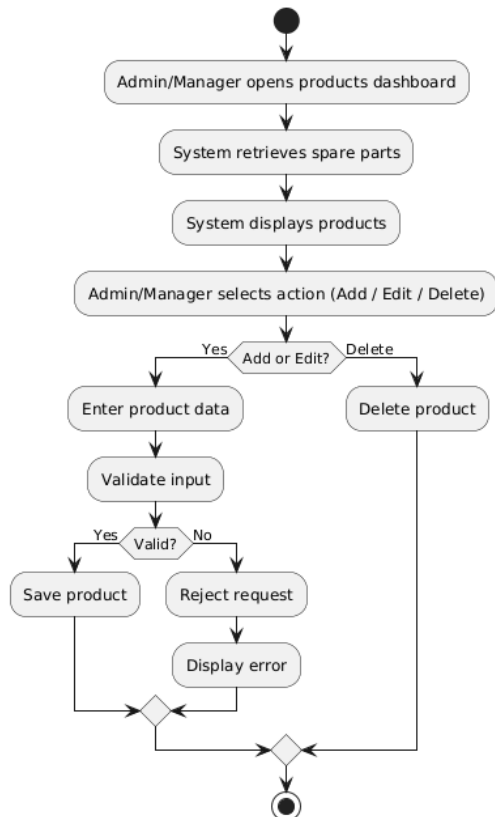
Manage Categories (U-017)



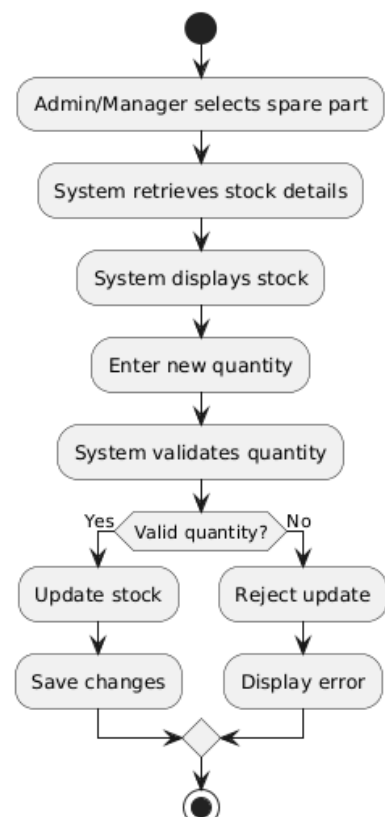
Manage Suppliers (U-018)



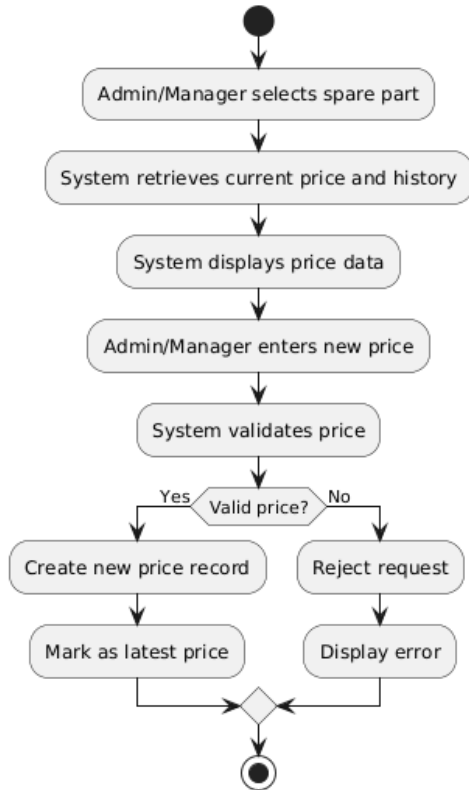
Manage Spare Parts (U-019)



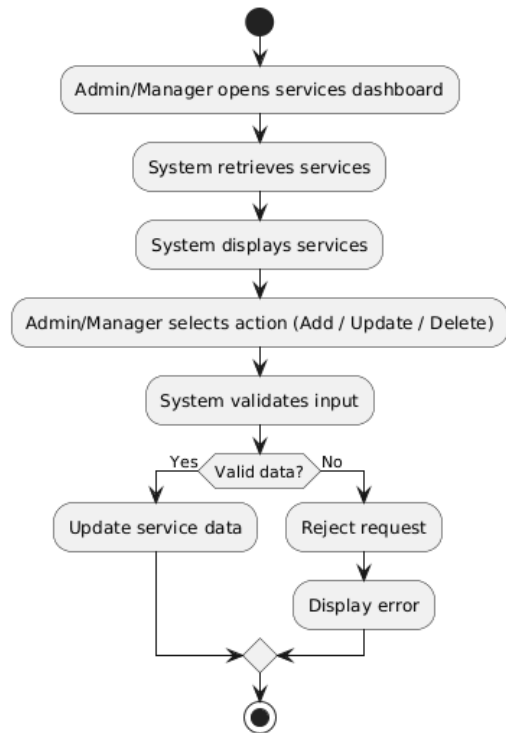
Manage Stock (U-020)



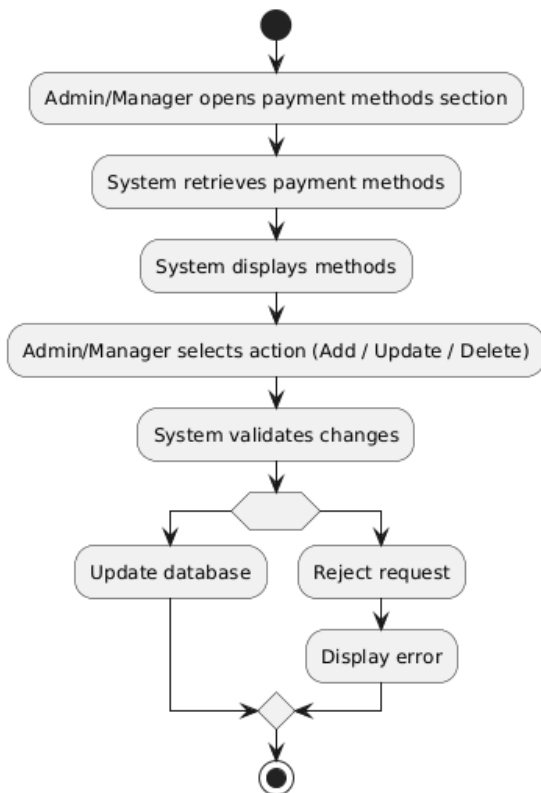
Manage Part Prices (U-021)



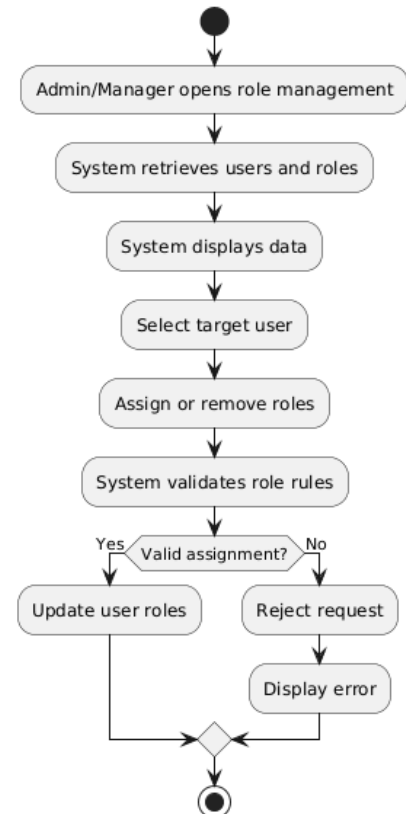
Manage Services (U-022)



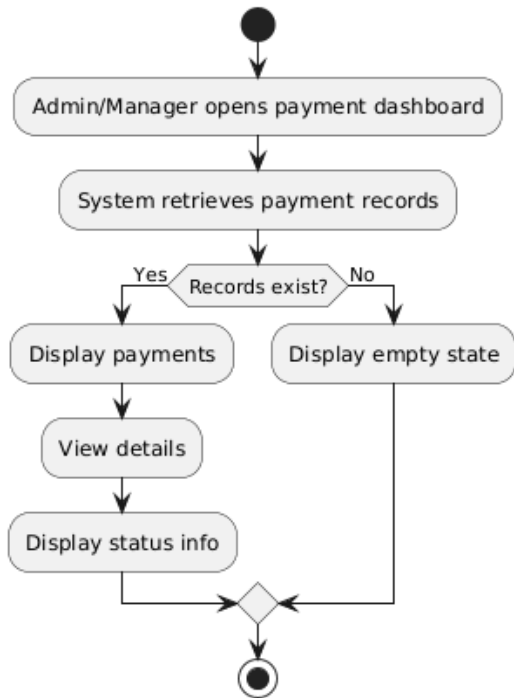
Manage Payment Methods (U-023)



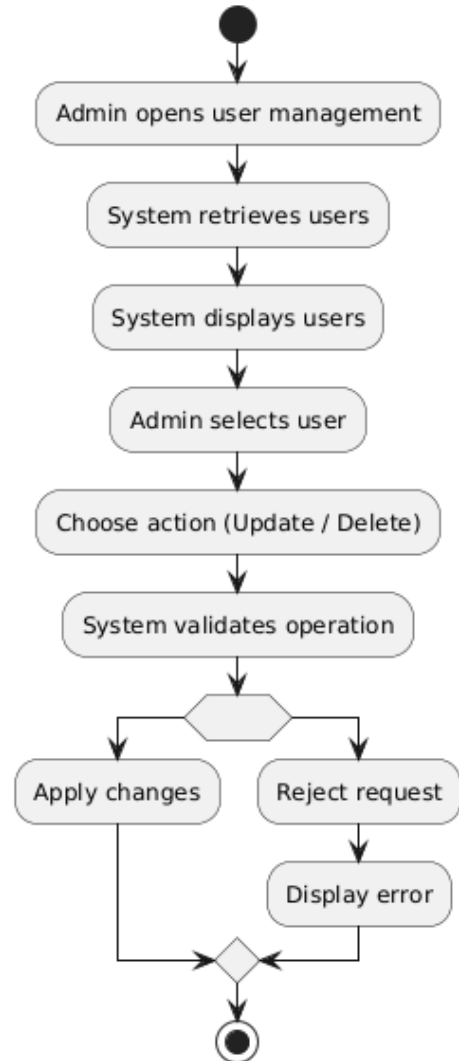
Manage User Roles (U-024)



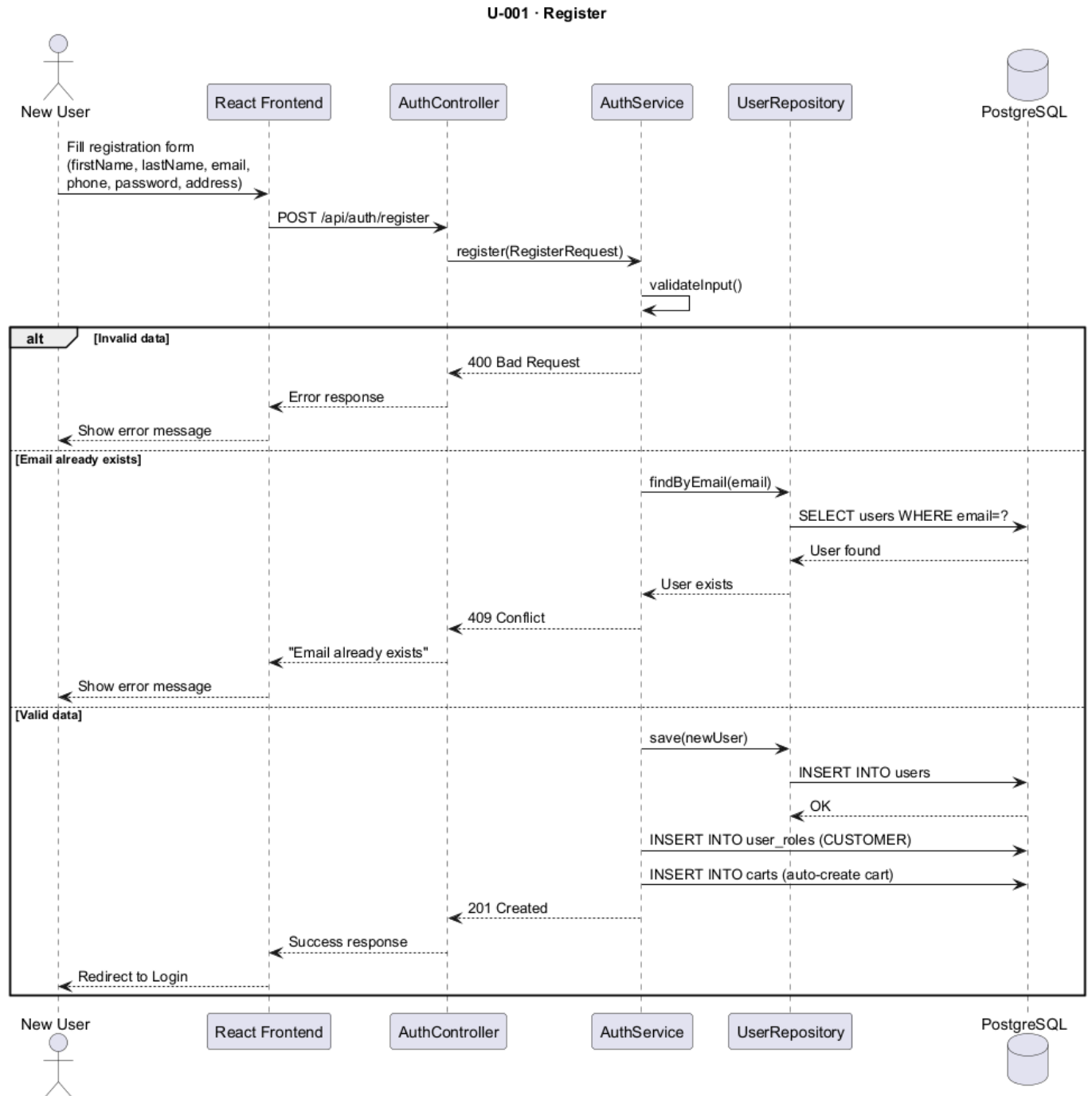
View Payments (U-025)



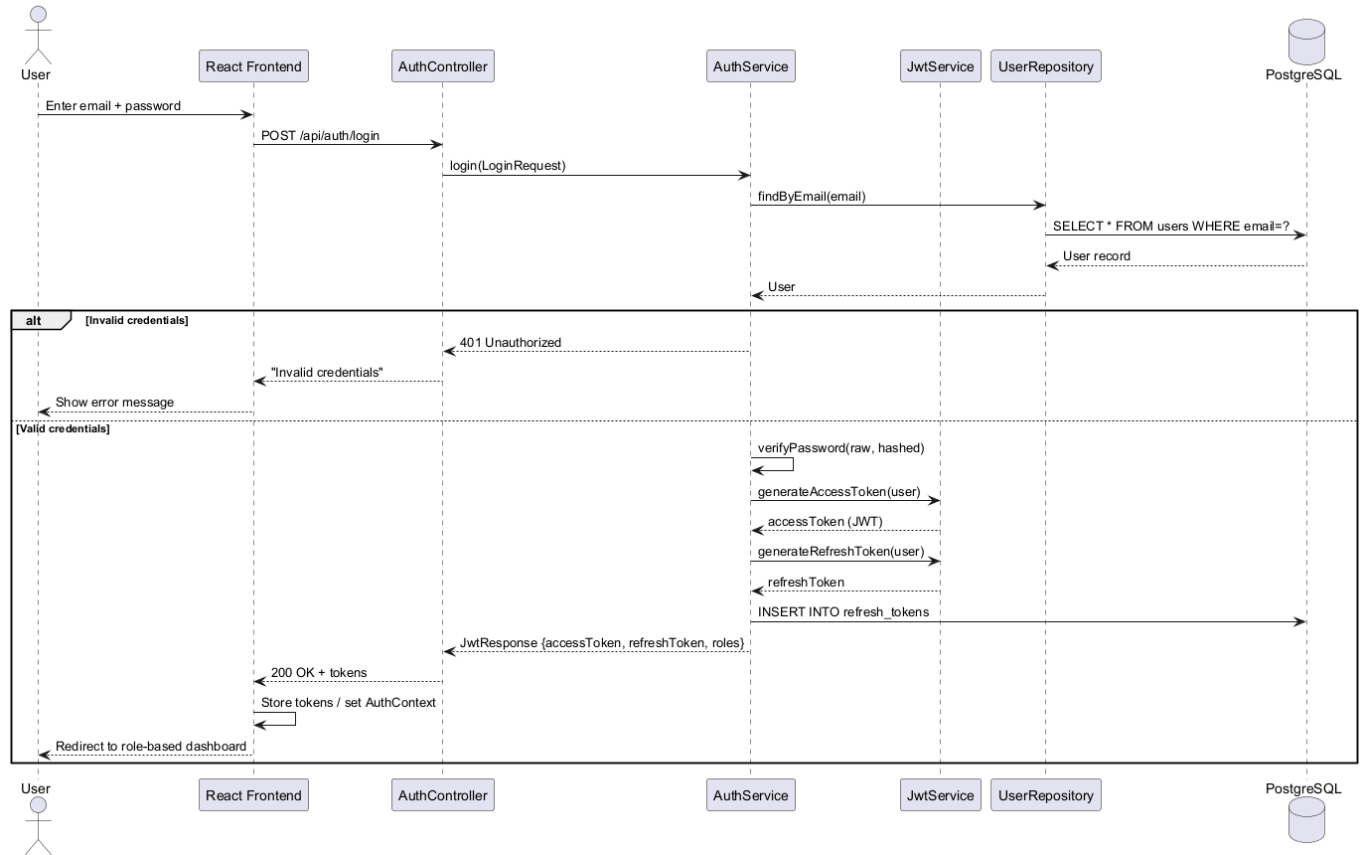
Manage Users (U-026)



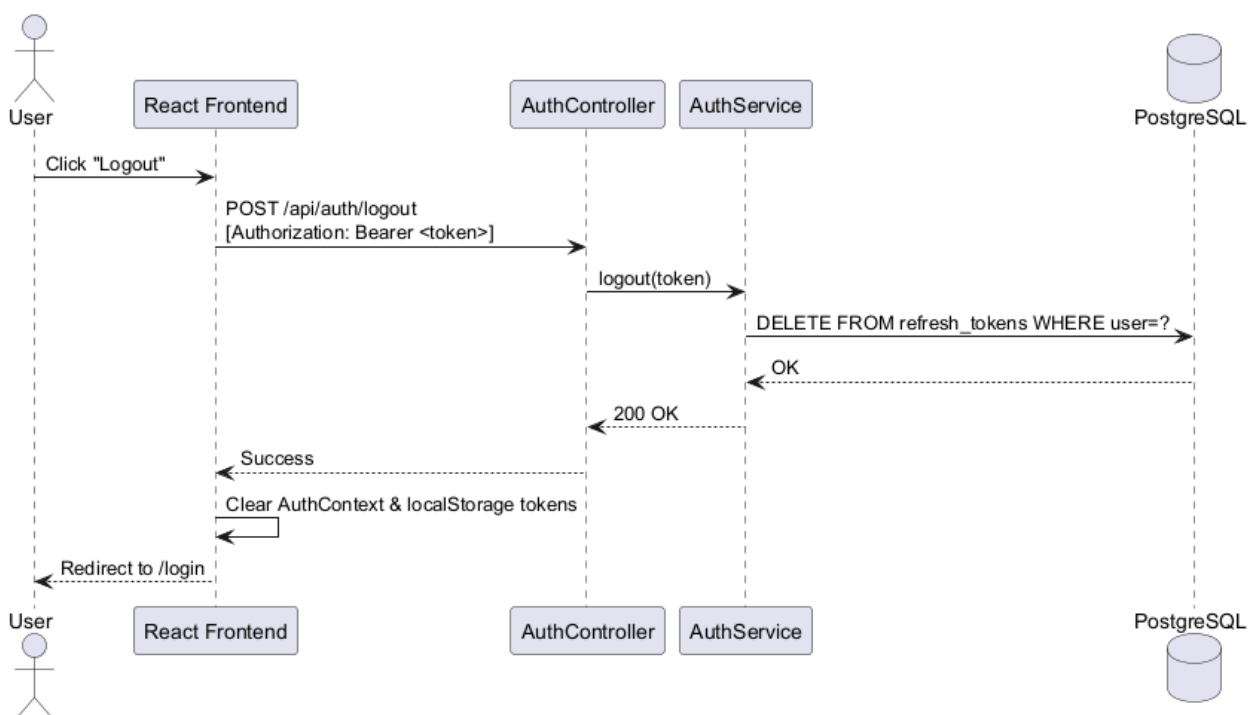
6.2 Sequence Diagrams:



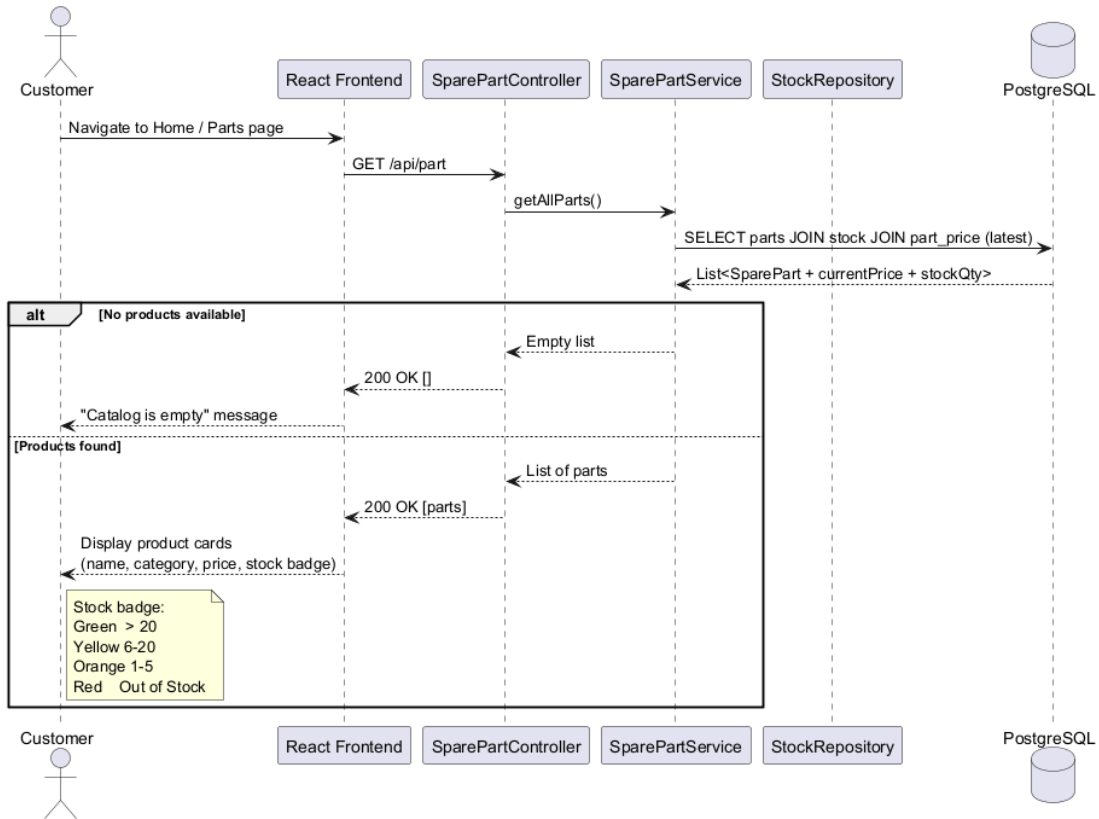
U-002 · Login



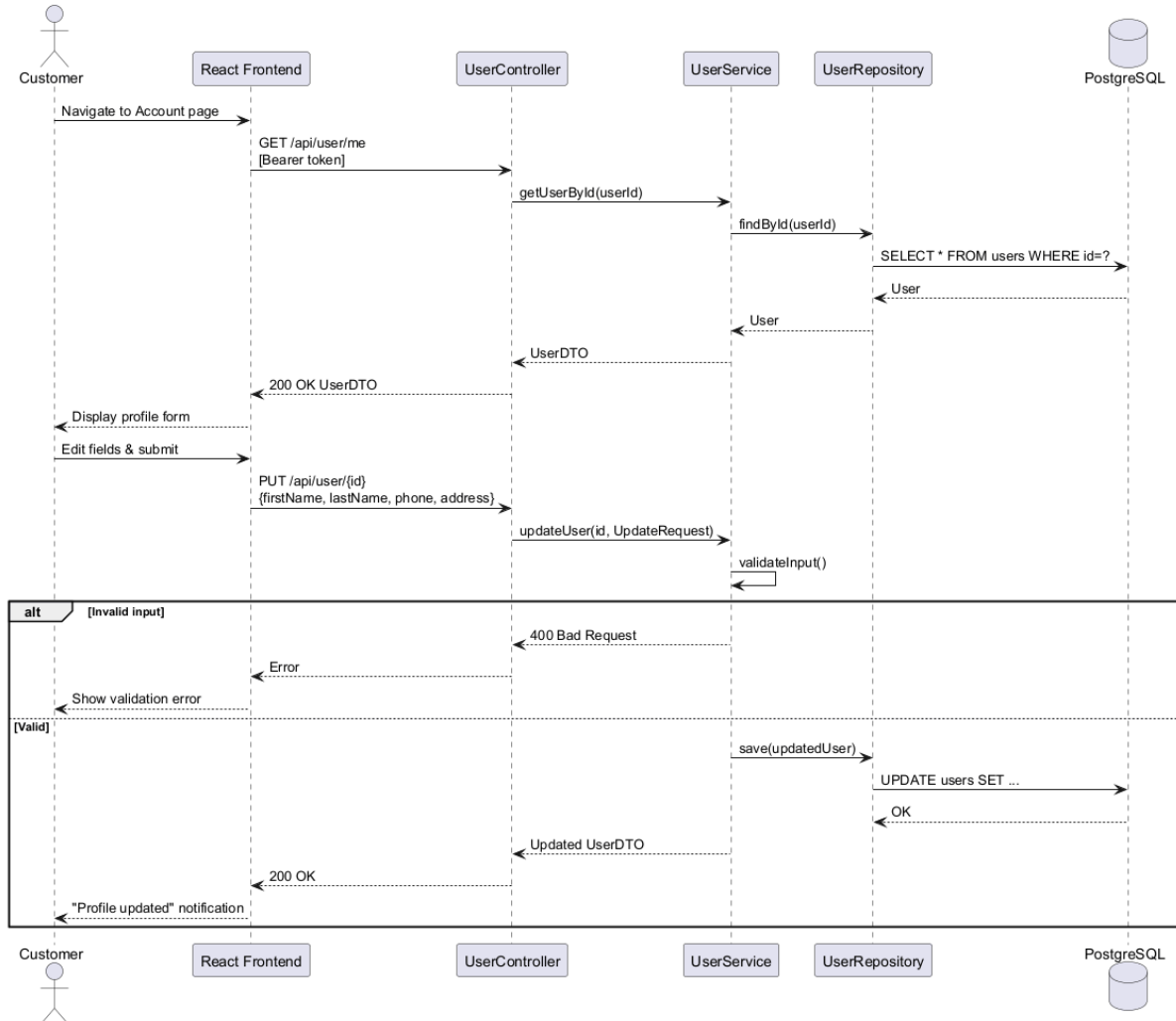
U-003 · Logout



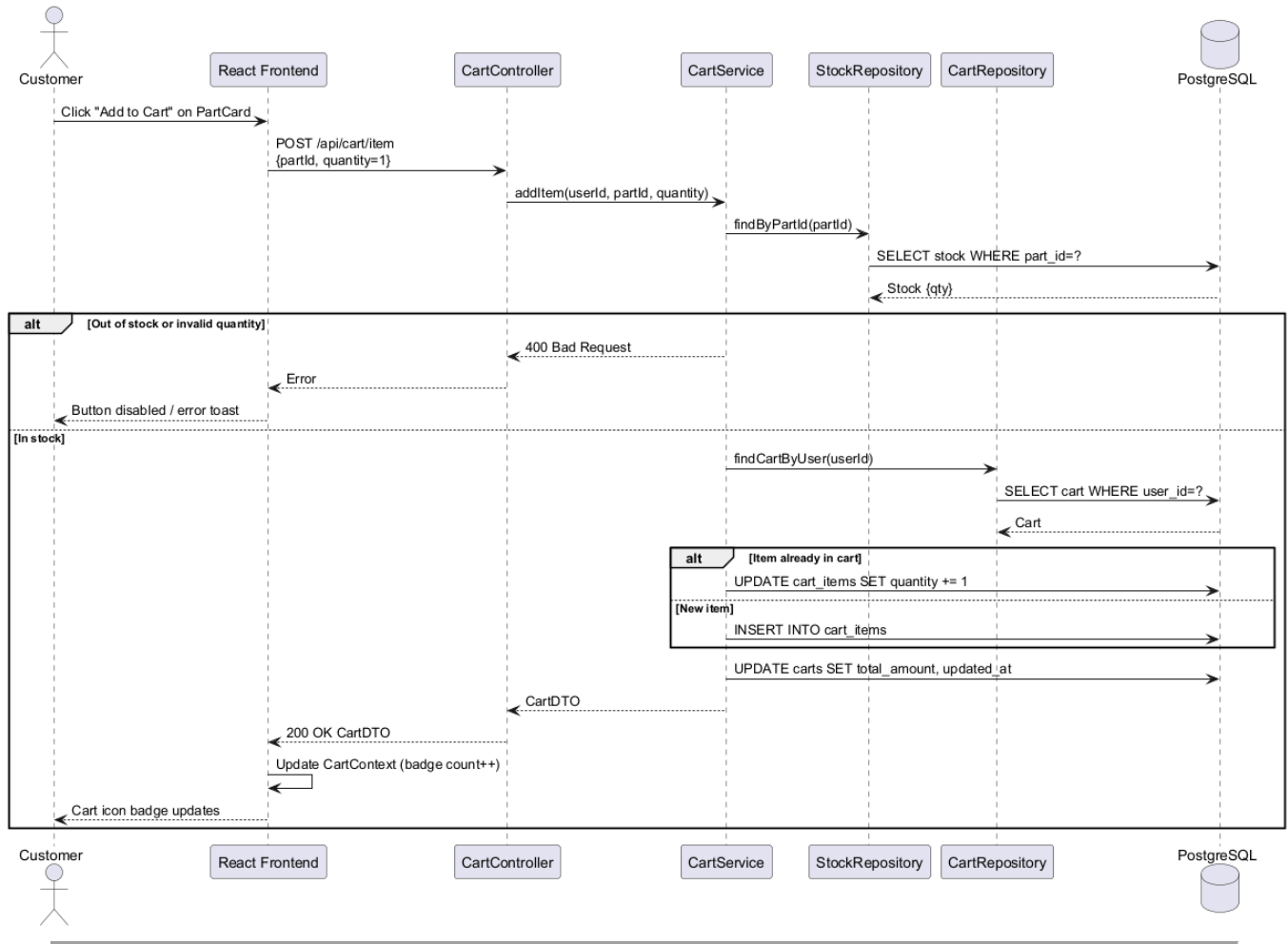
U-004 · View Spare Parts



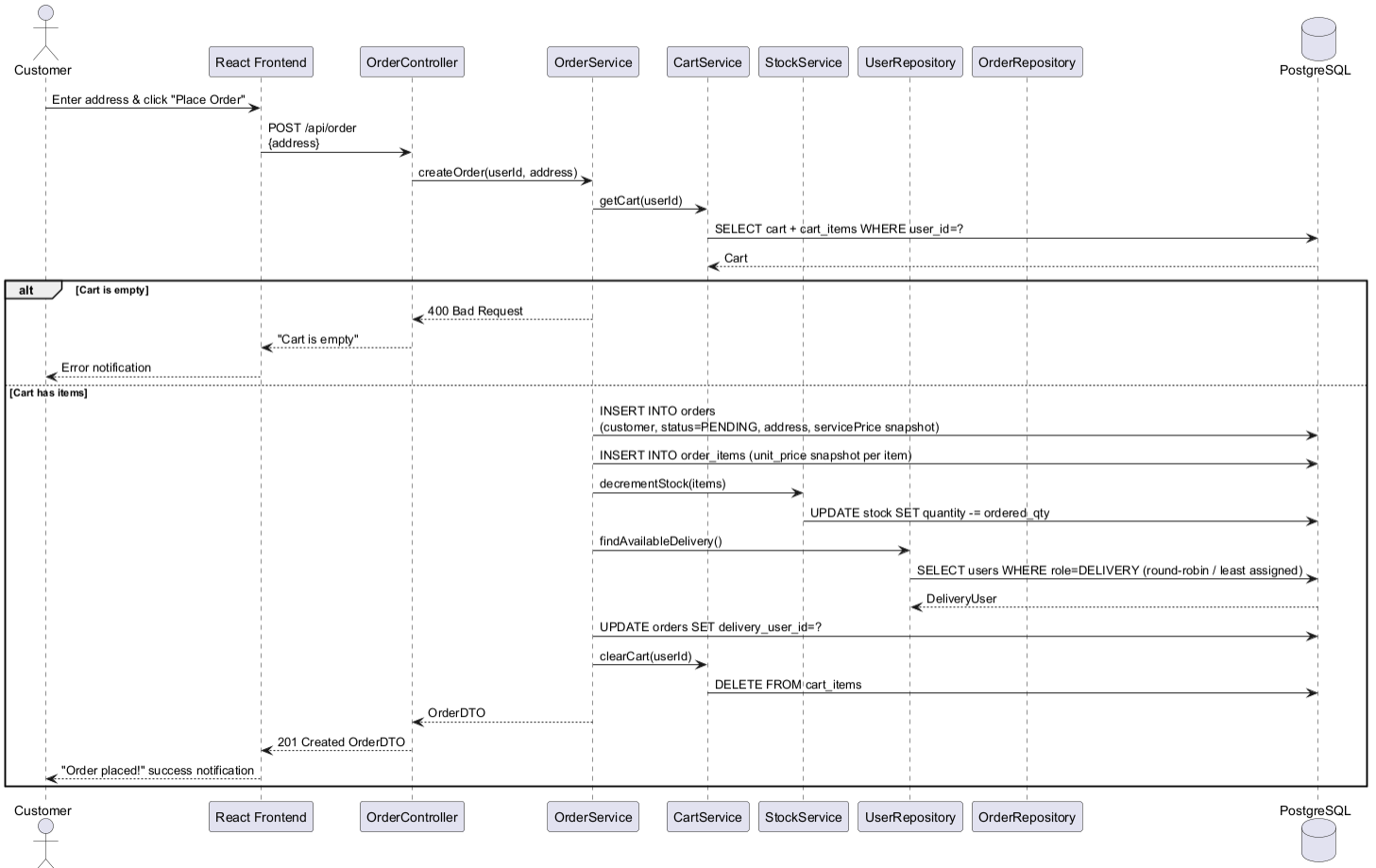
U-005 · Manage Account



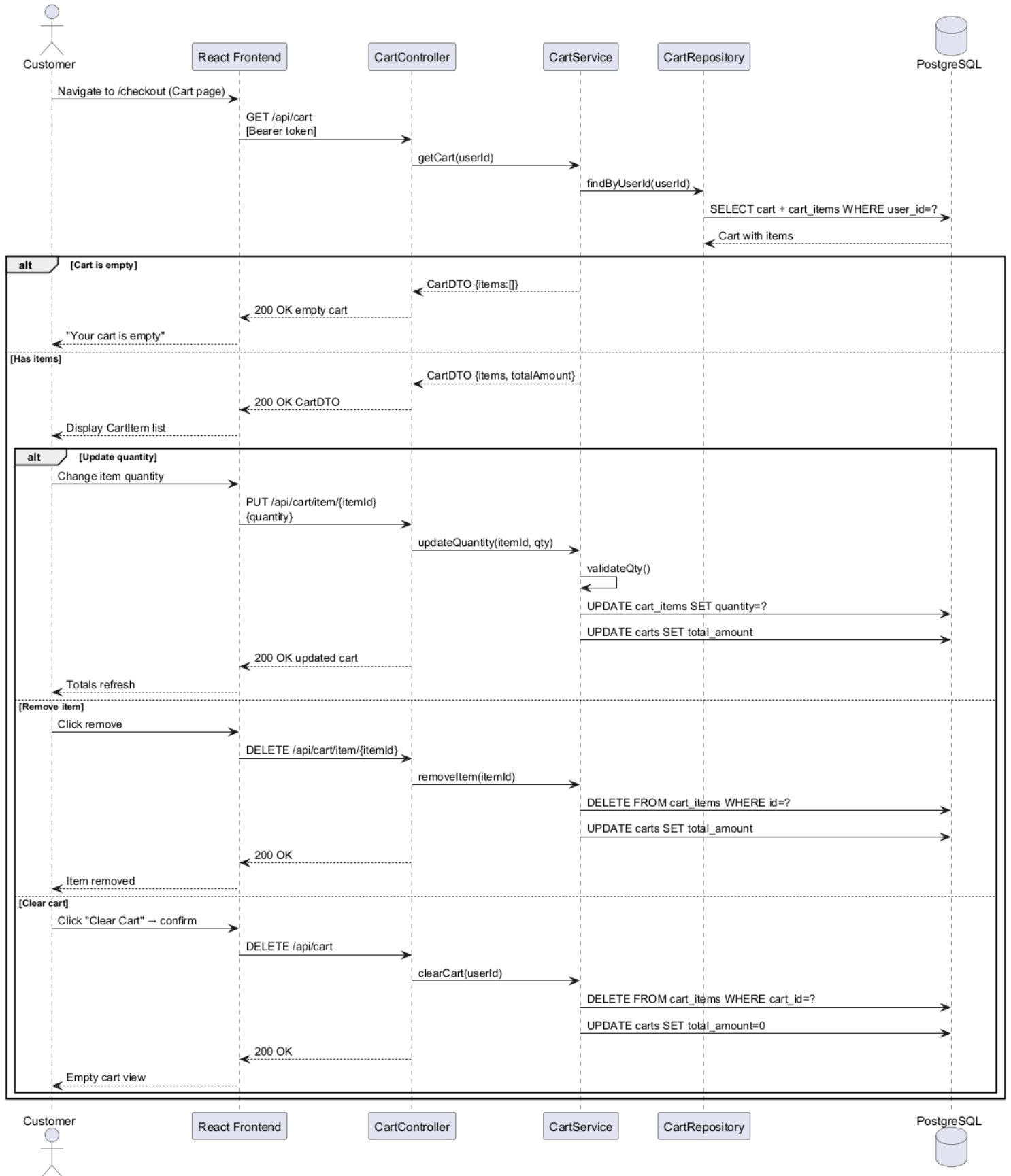
U-006 · Add Item to Cart



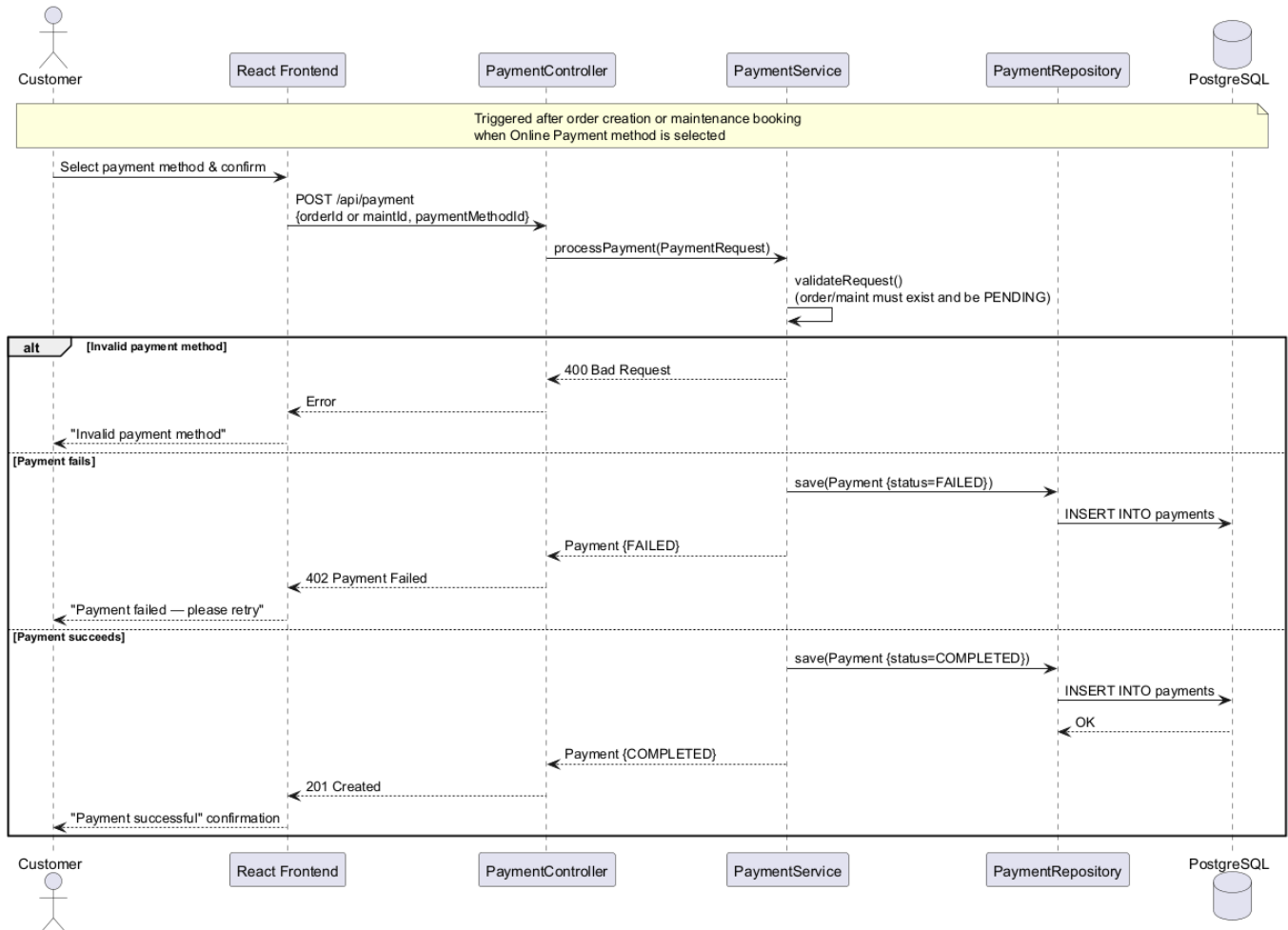
U-008 · Checkout (Create Order)



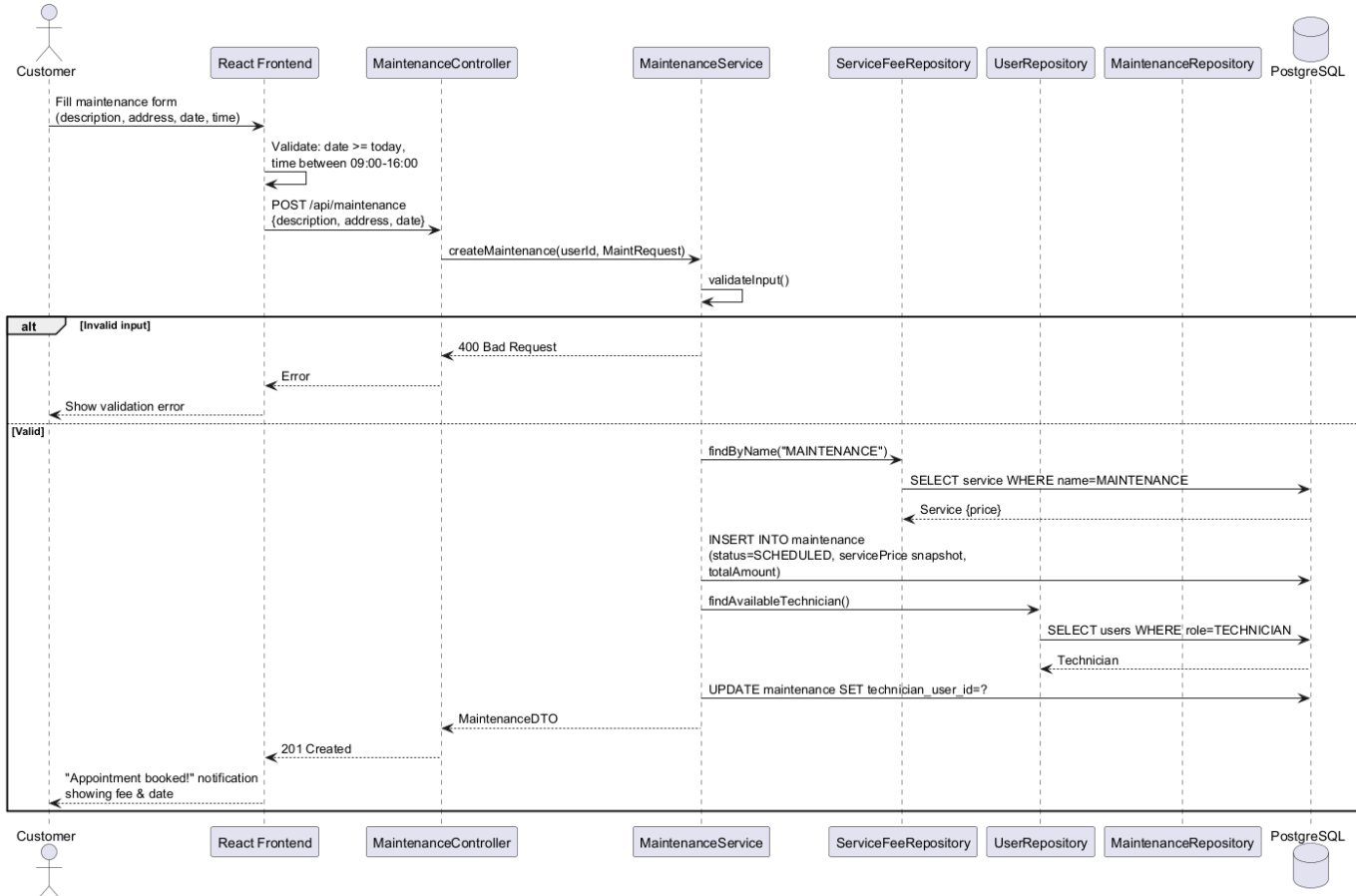
U-007 · Manage Cart



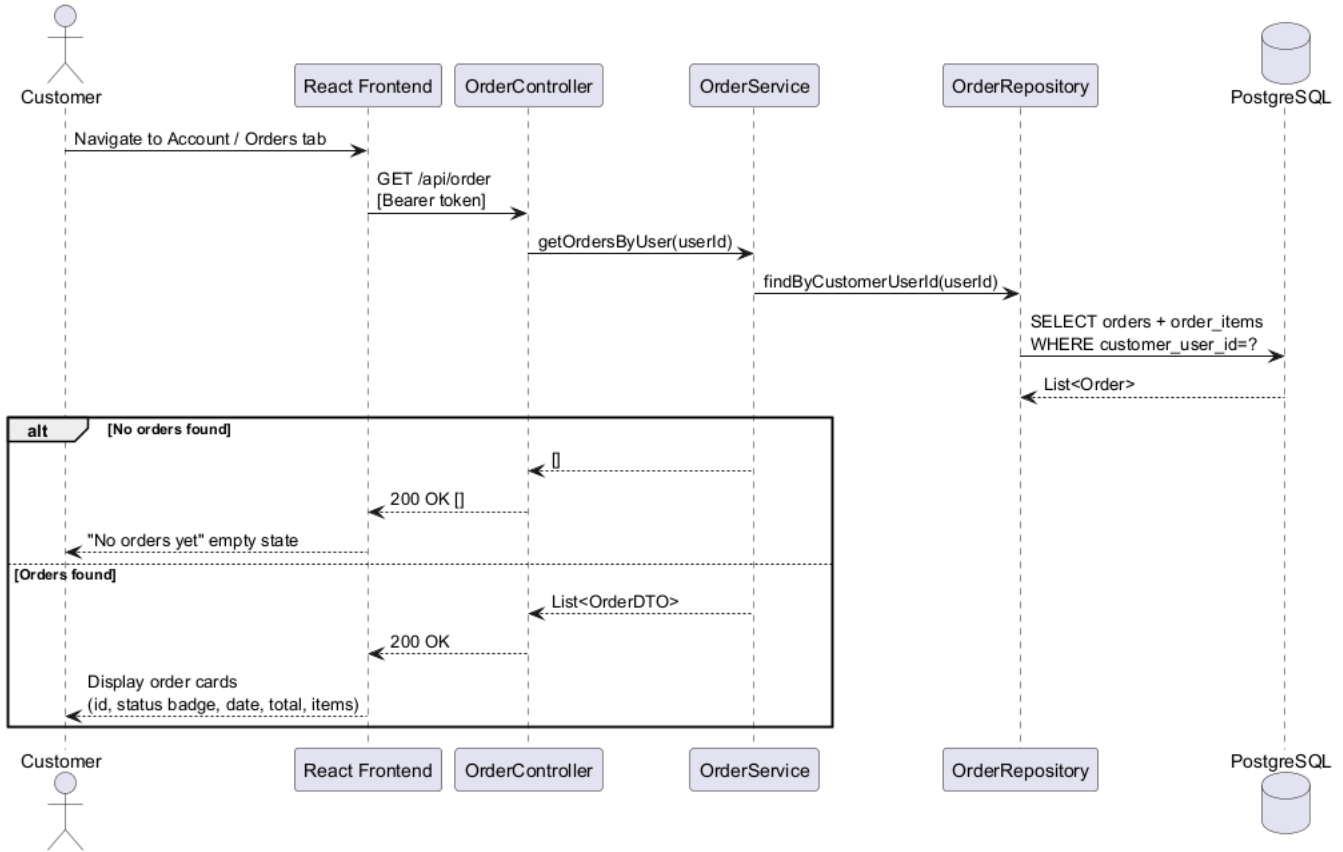
U-009 · Make Payment



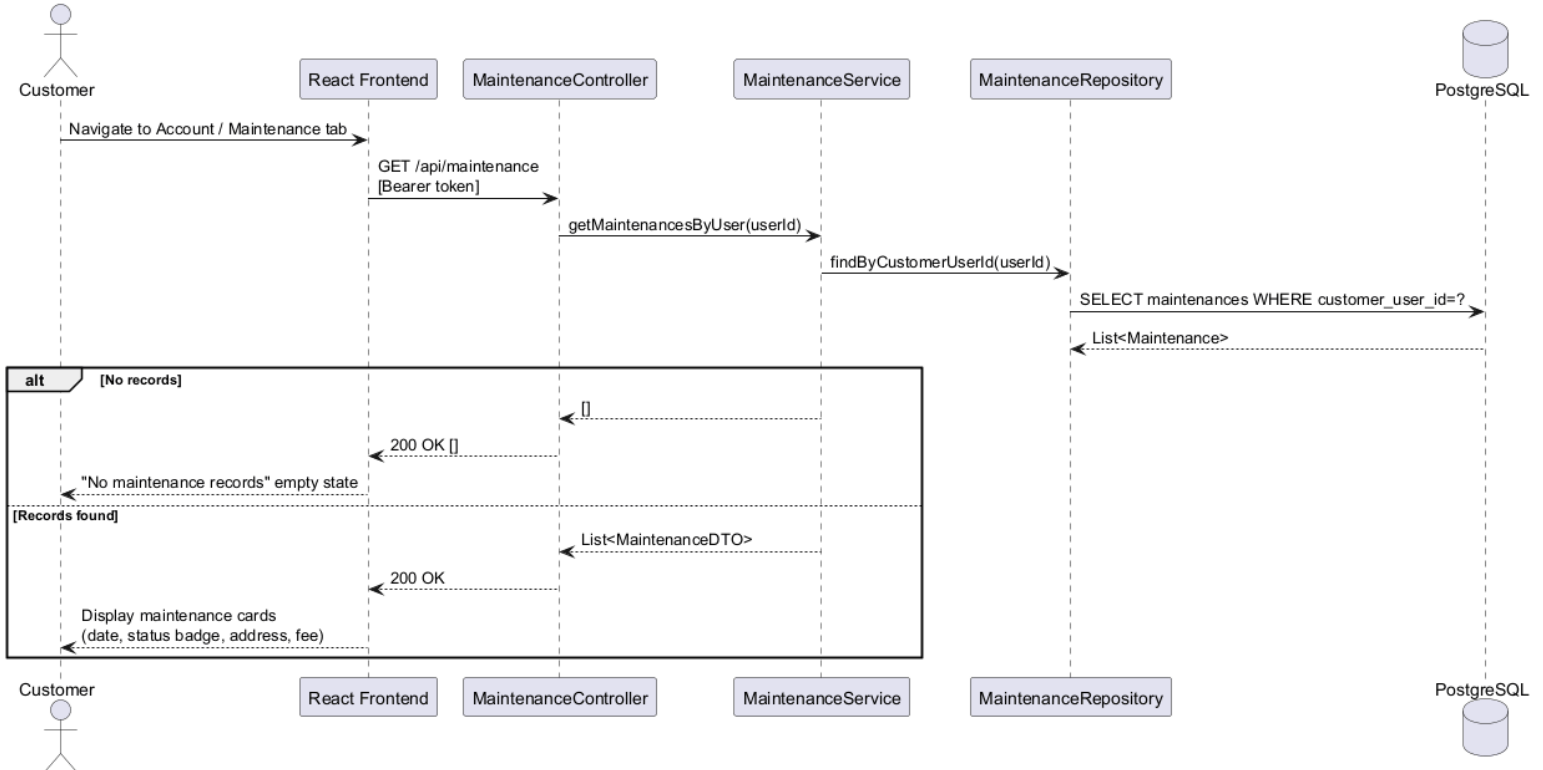
U-010 · Book Maintenance



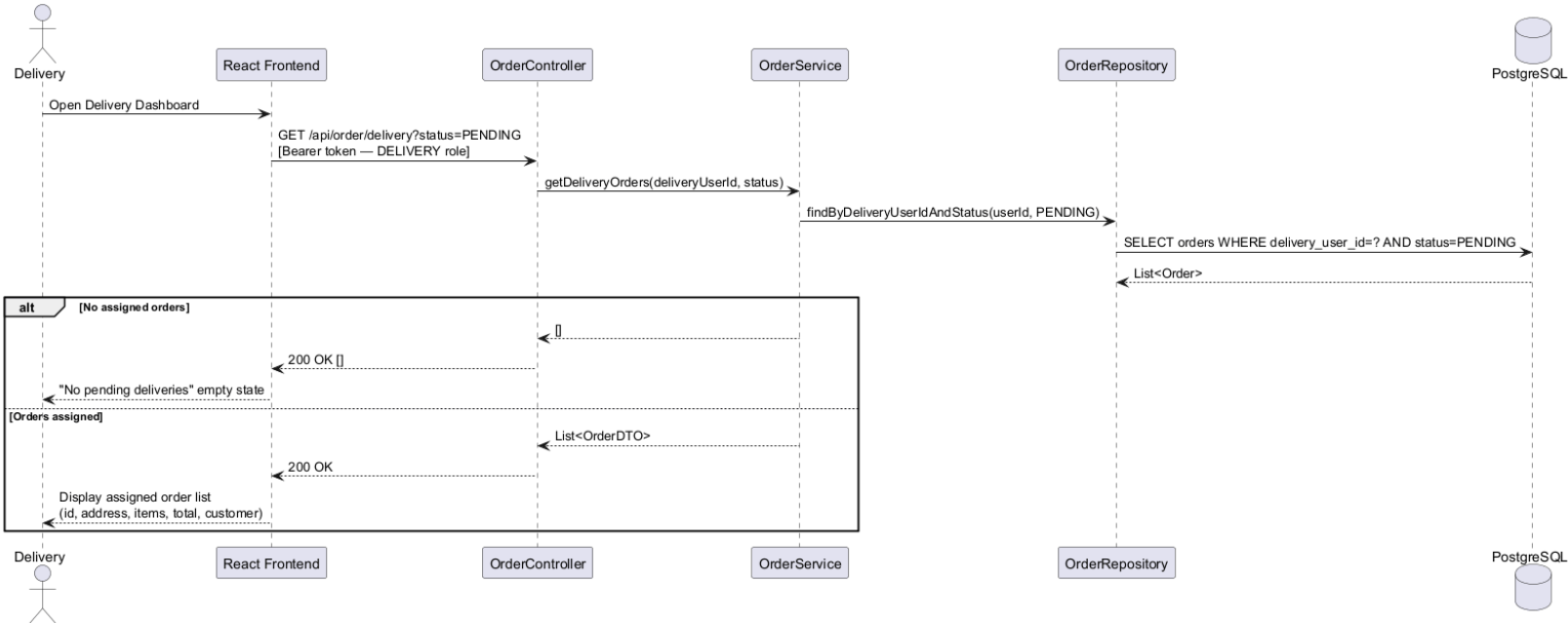
U-011 · View Orders



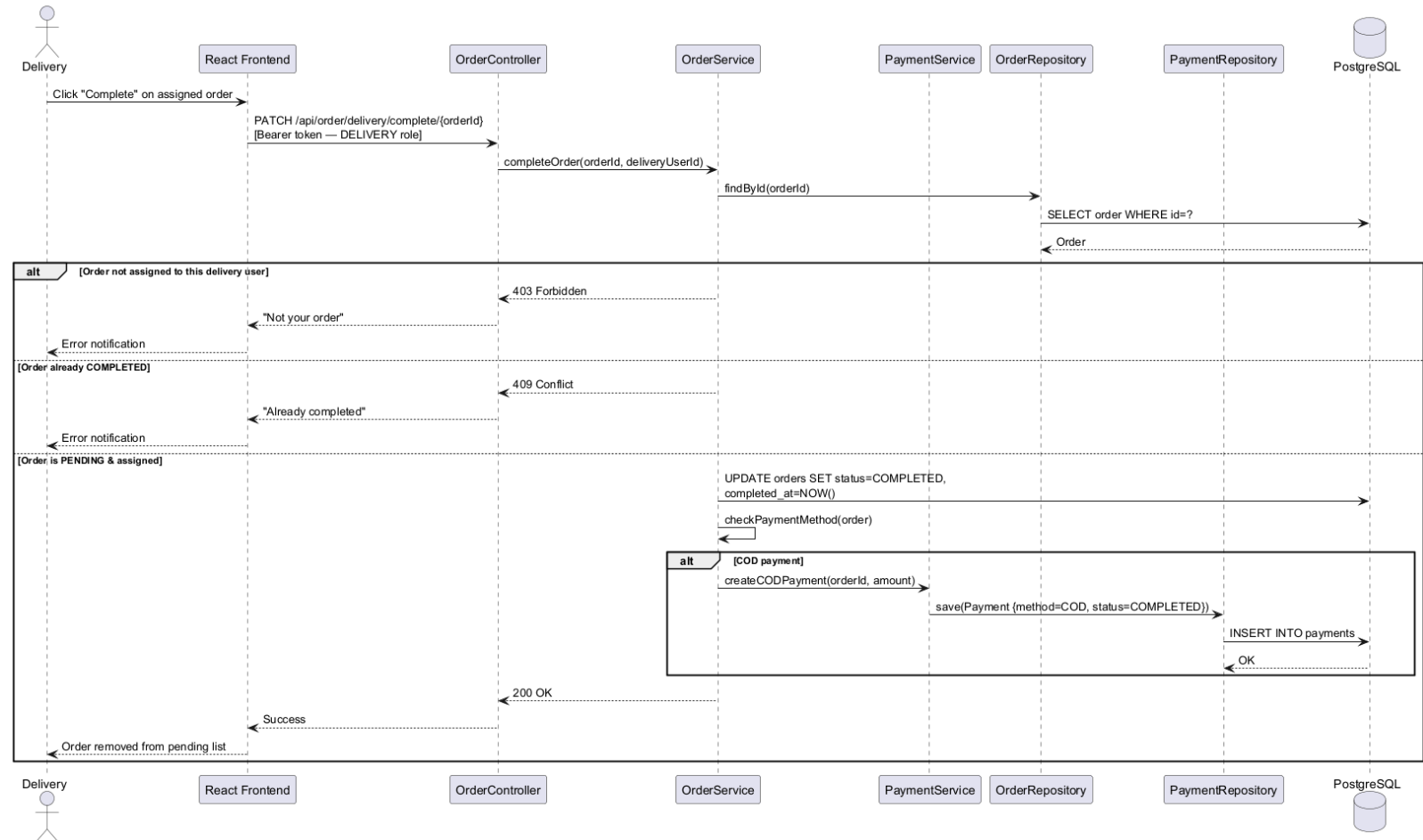
U-012 · View Maintenance



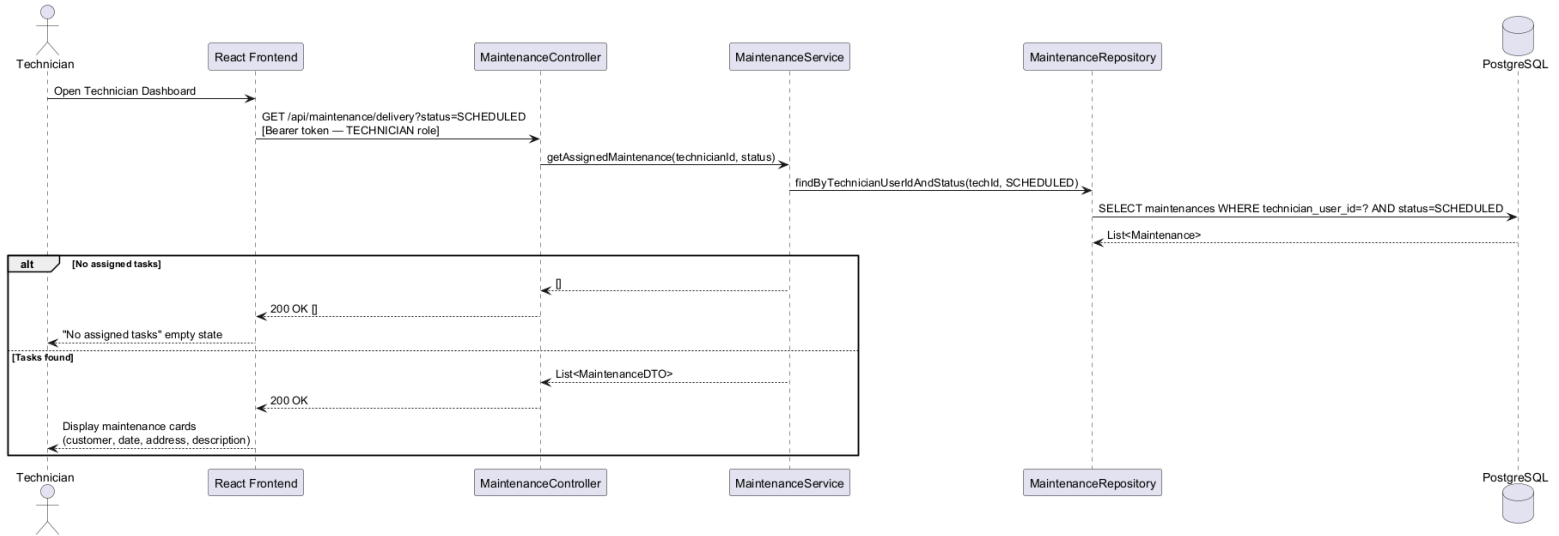
U-013 · View Assigned Orders



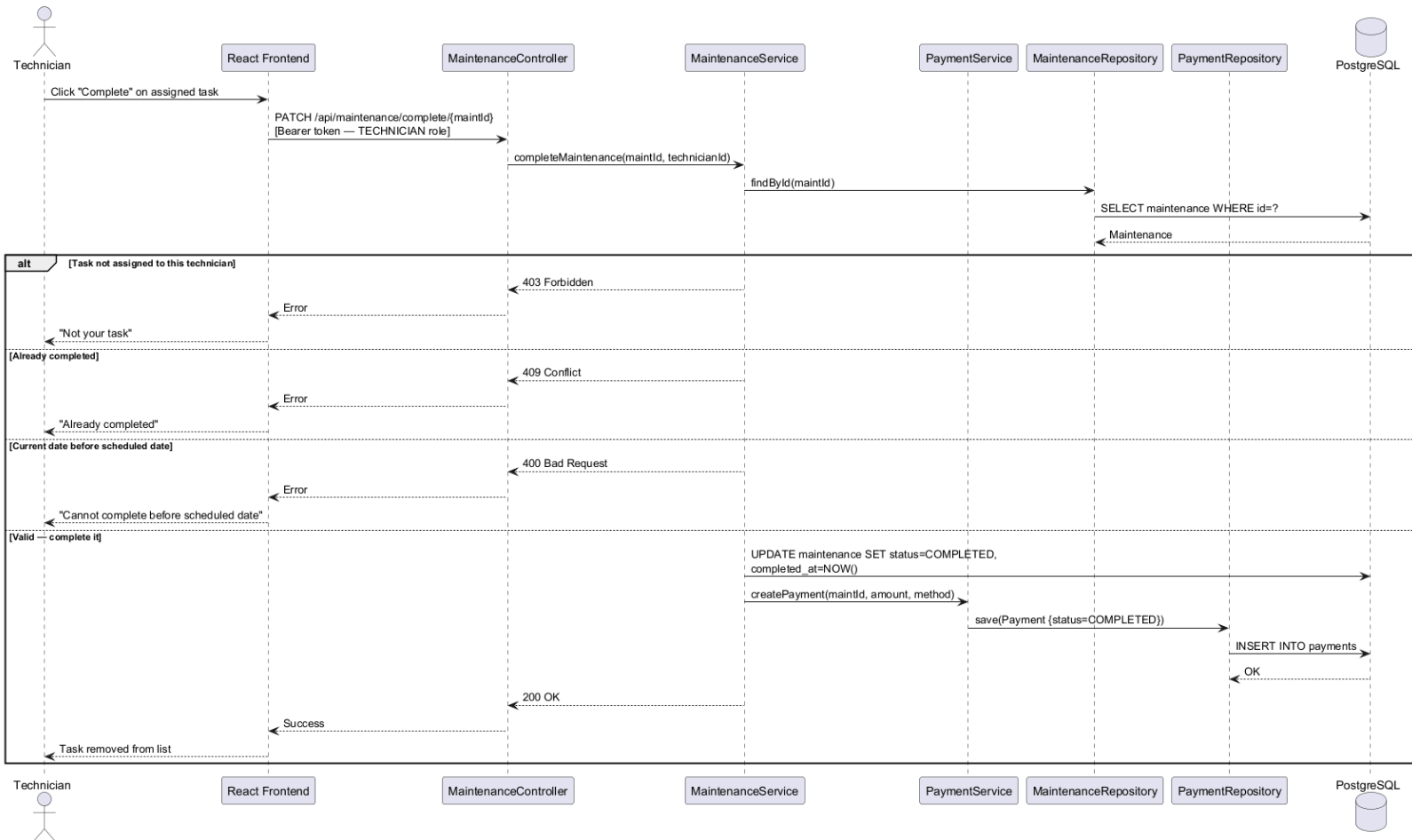
U-014 · Complete Delivery



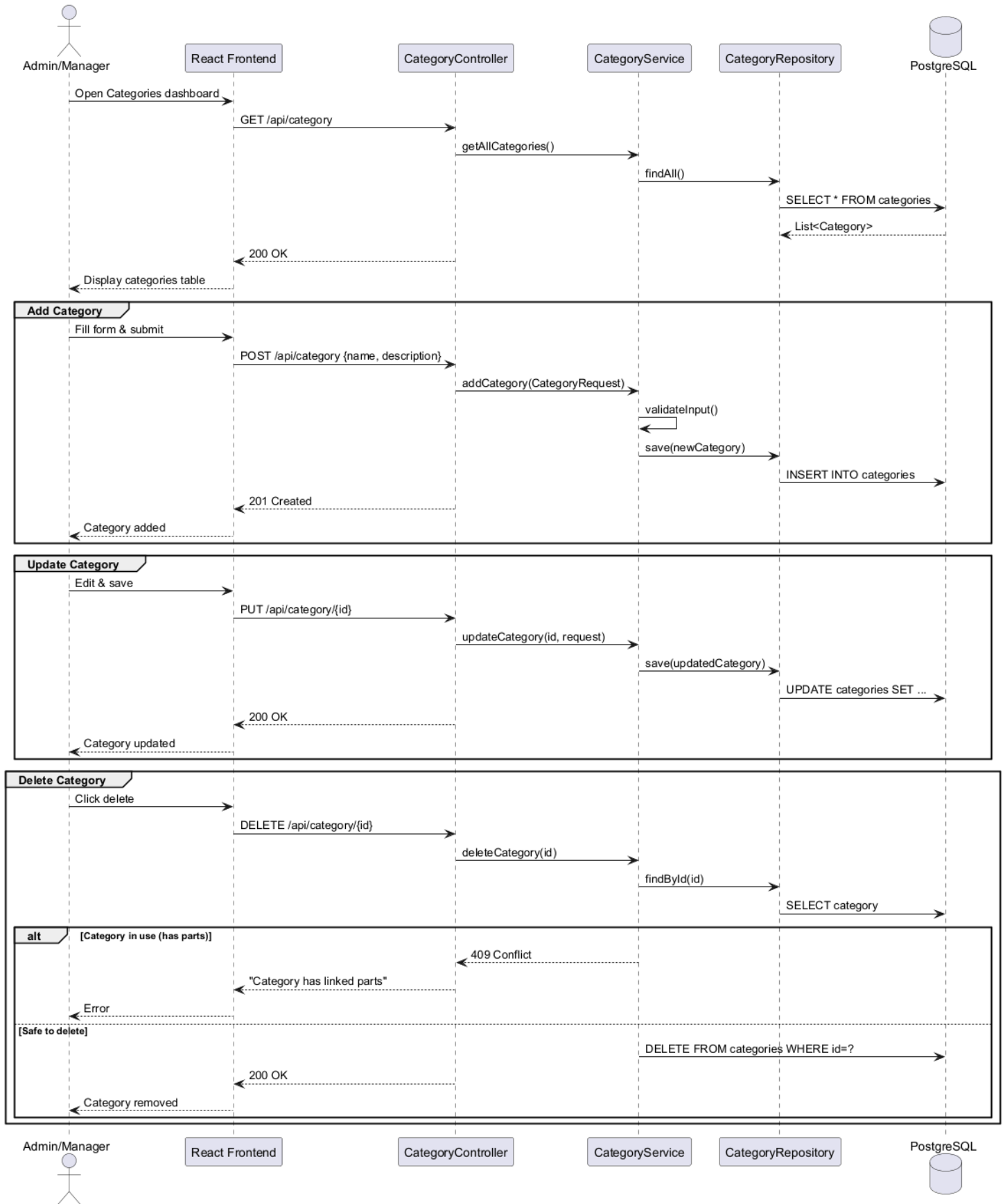
U-015 · View Assigned Maintenance



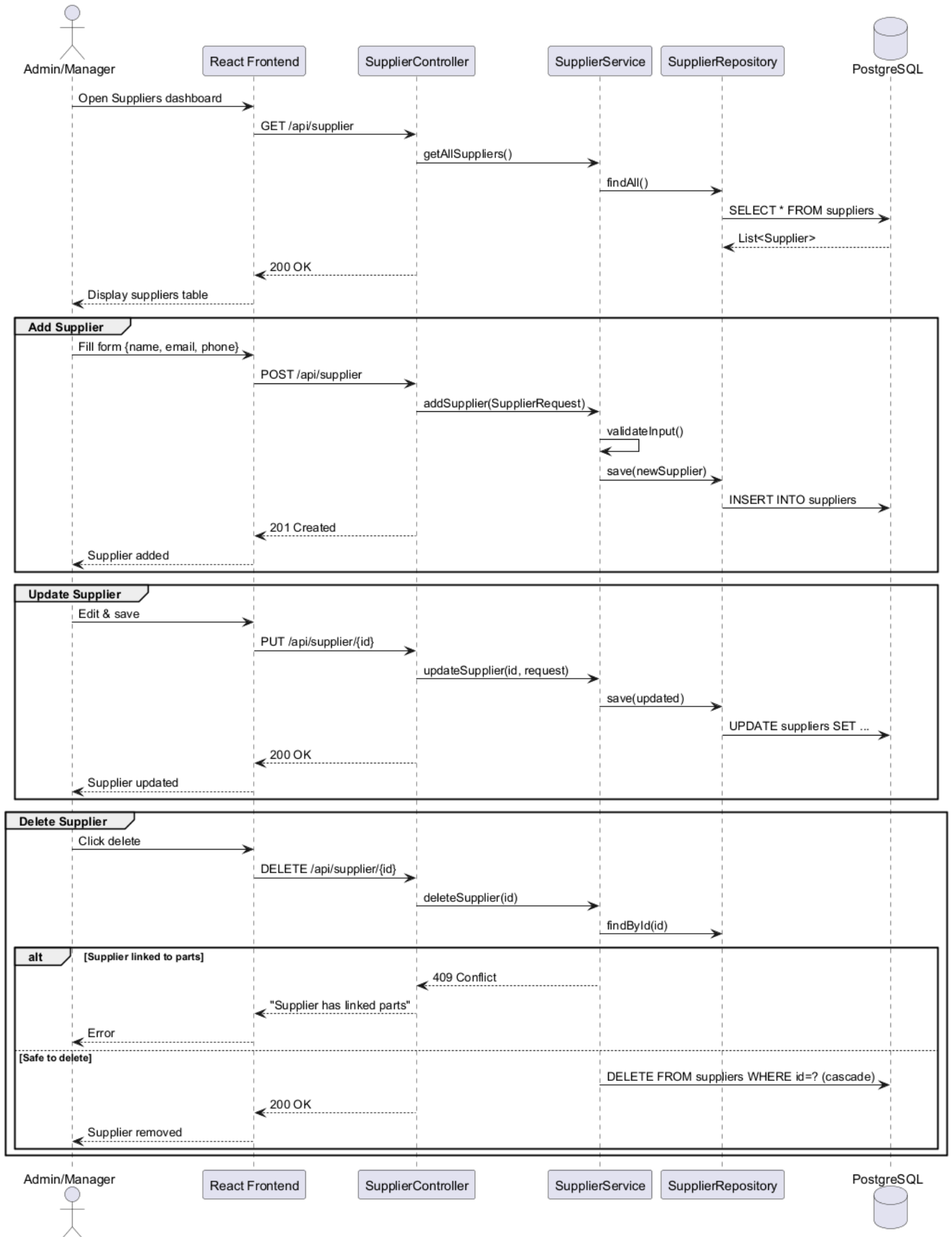
U-016 · Complete Maintenance



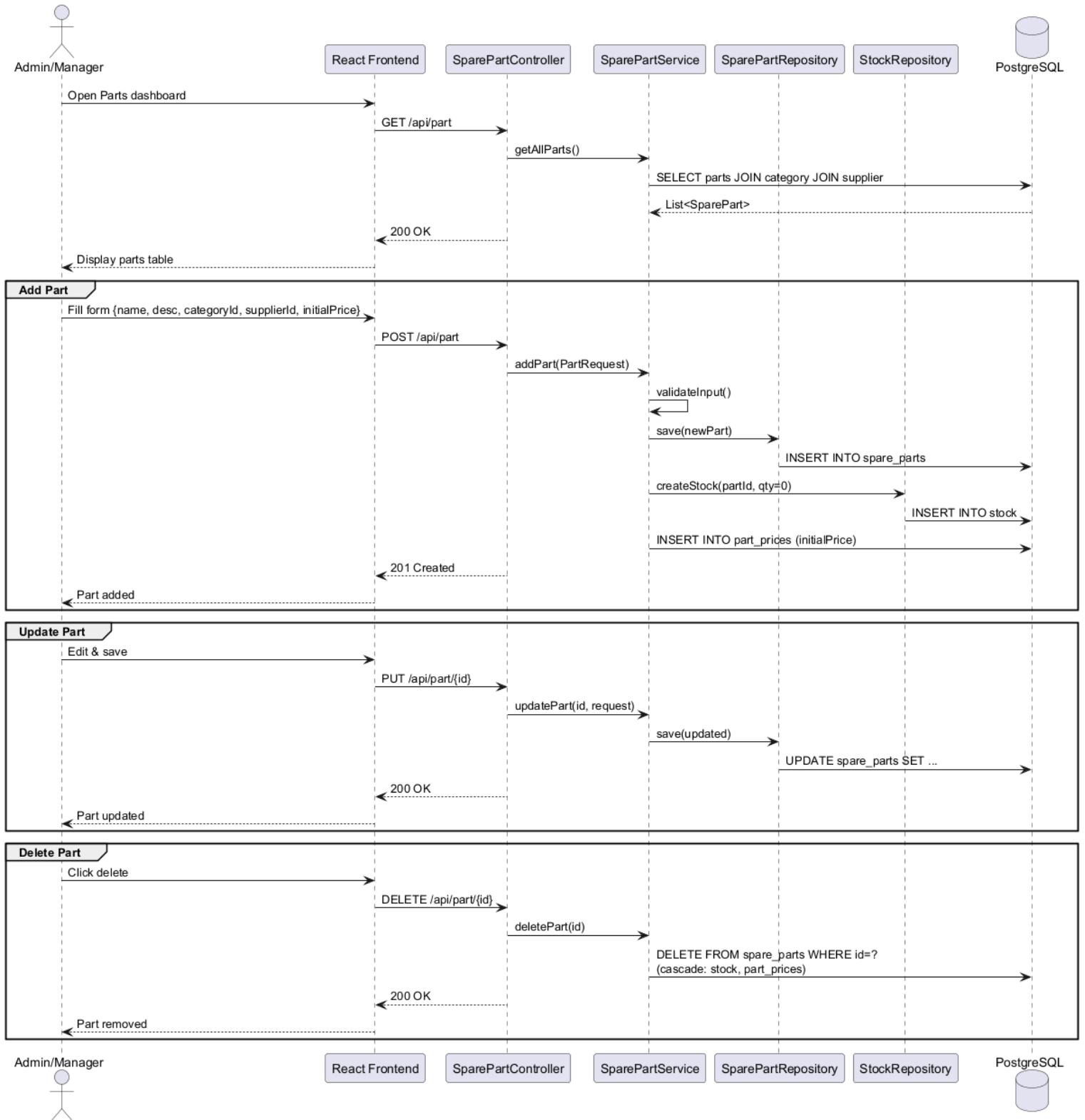
U-017 · Manage Categories (Admin / Manager)



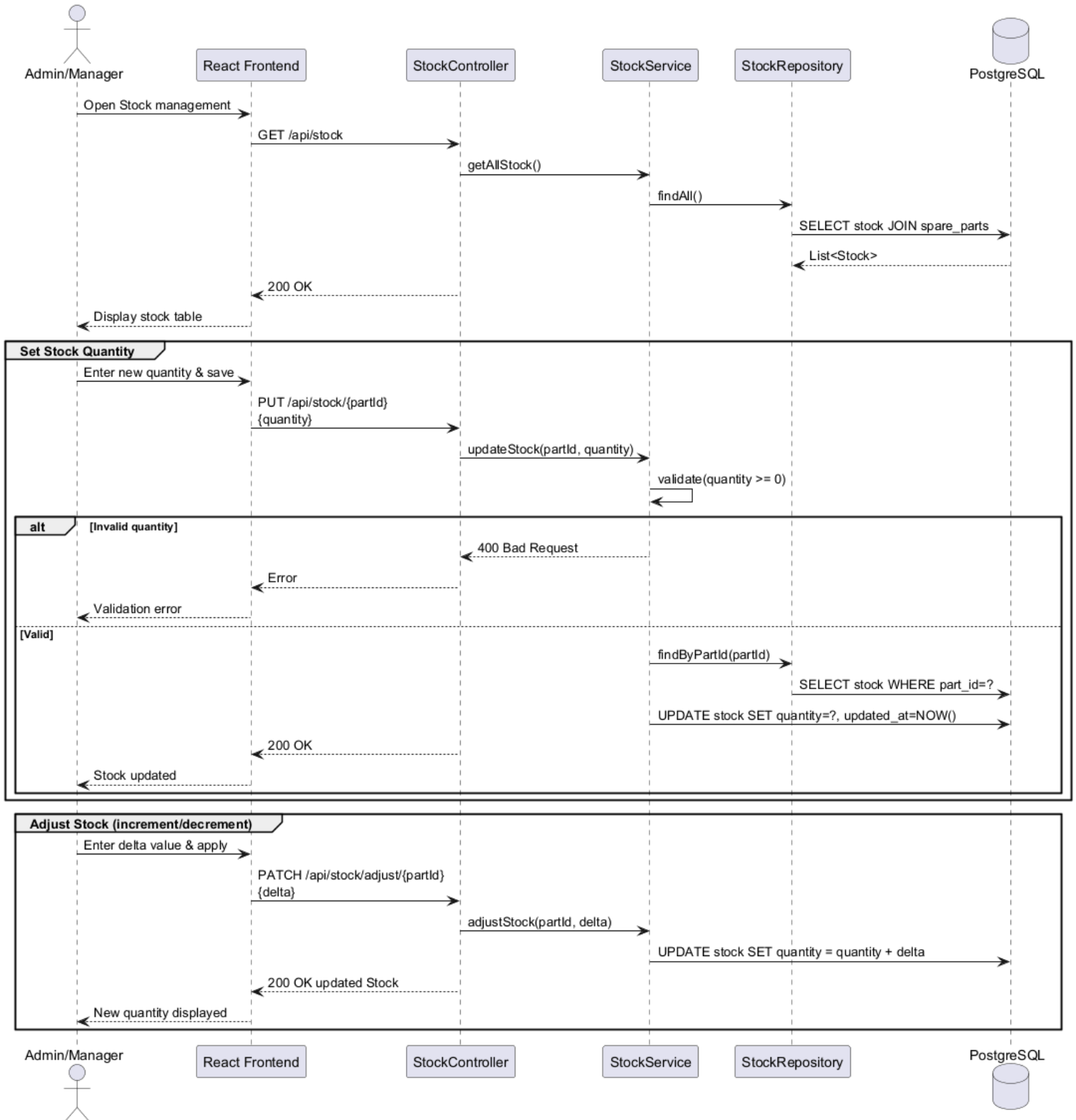
U-018 · Manage Suppliers (Admin / Manager)



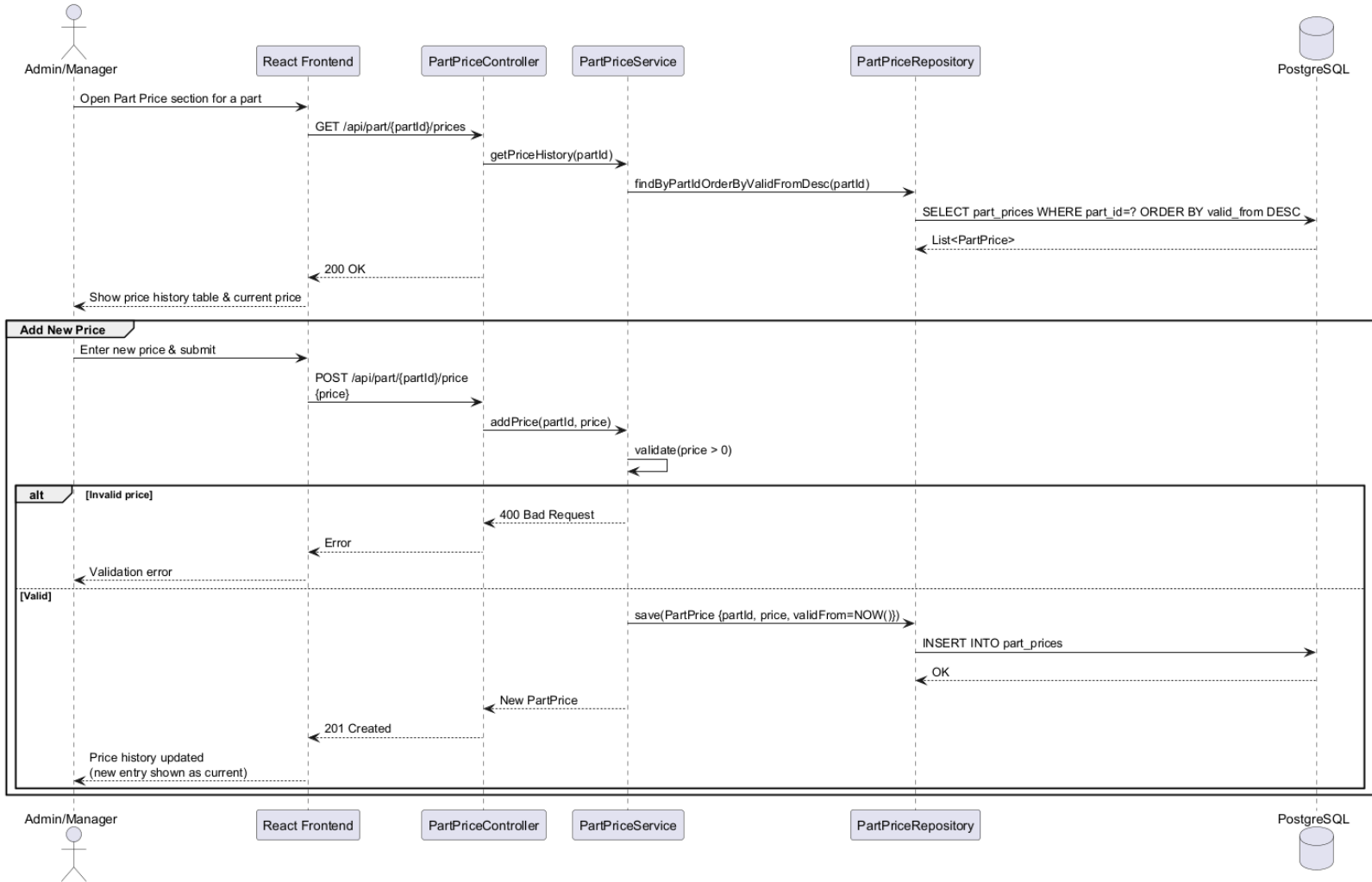
U-019 · Manage Spare Parts (Admin / Manager)



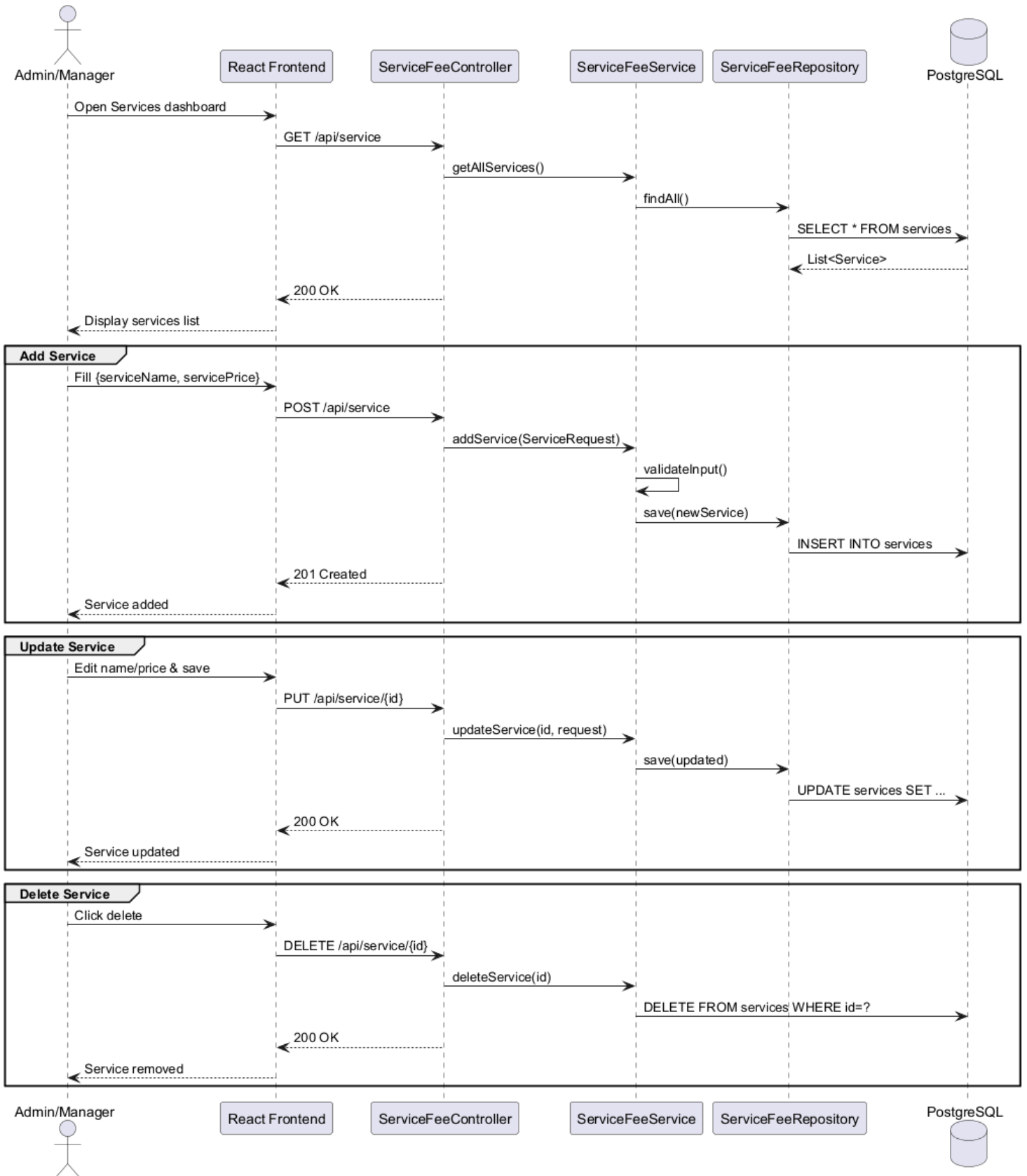
U-020 · Manage Stock (Admin / Manager)



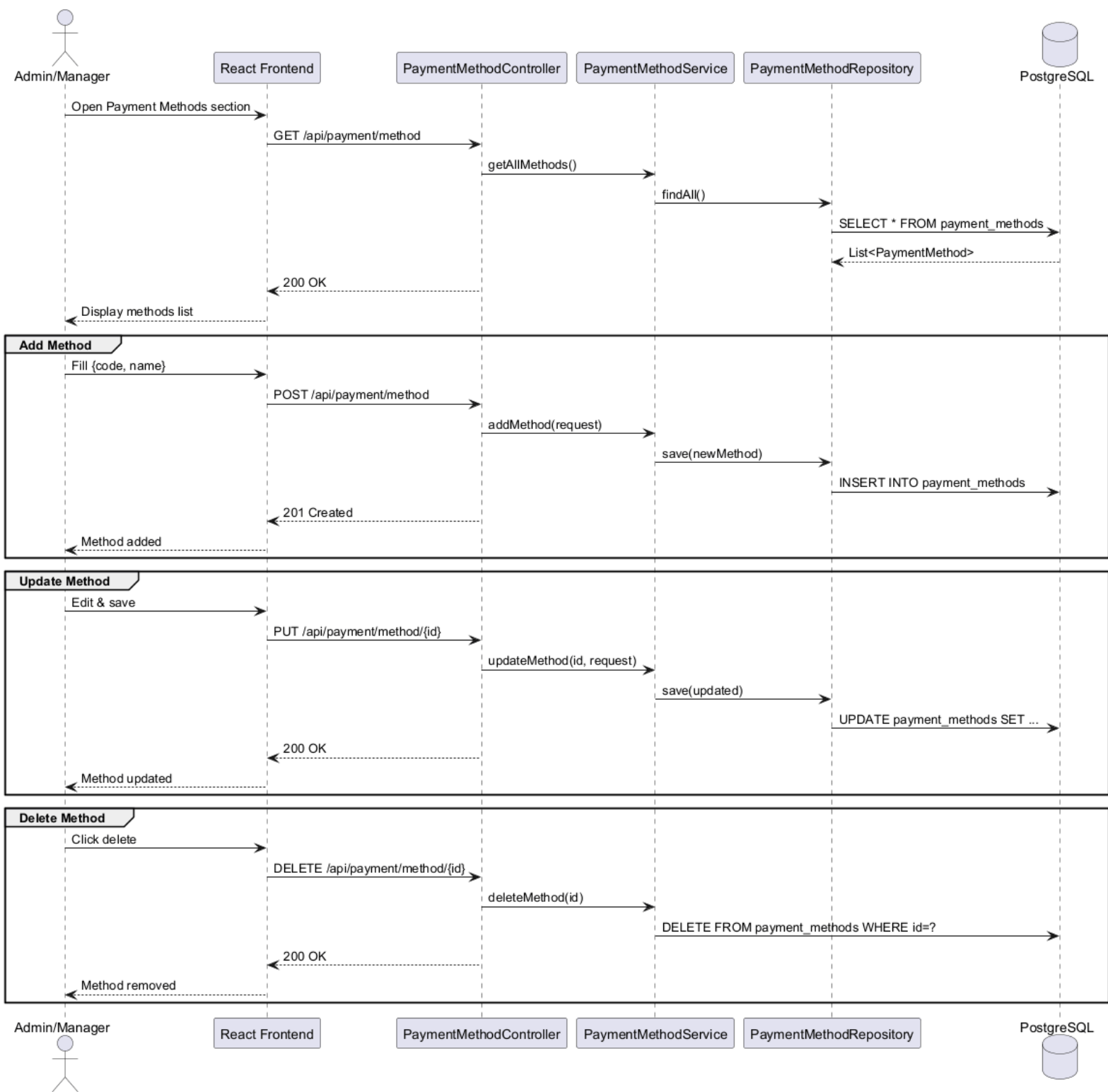
U-021 · Manage Part Prices (Admin / Manager)



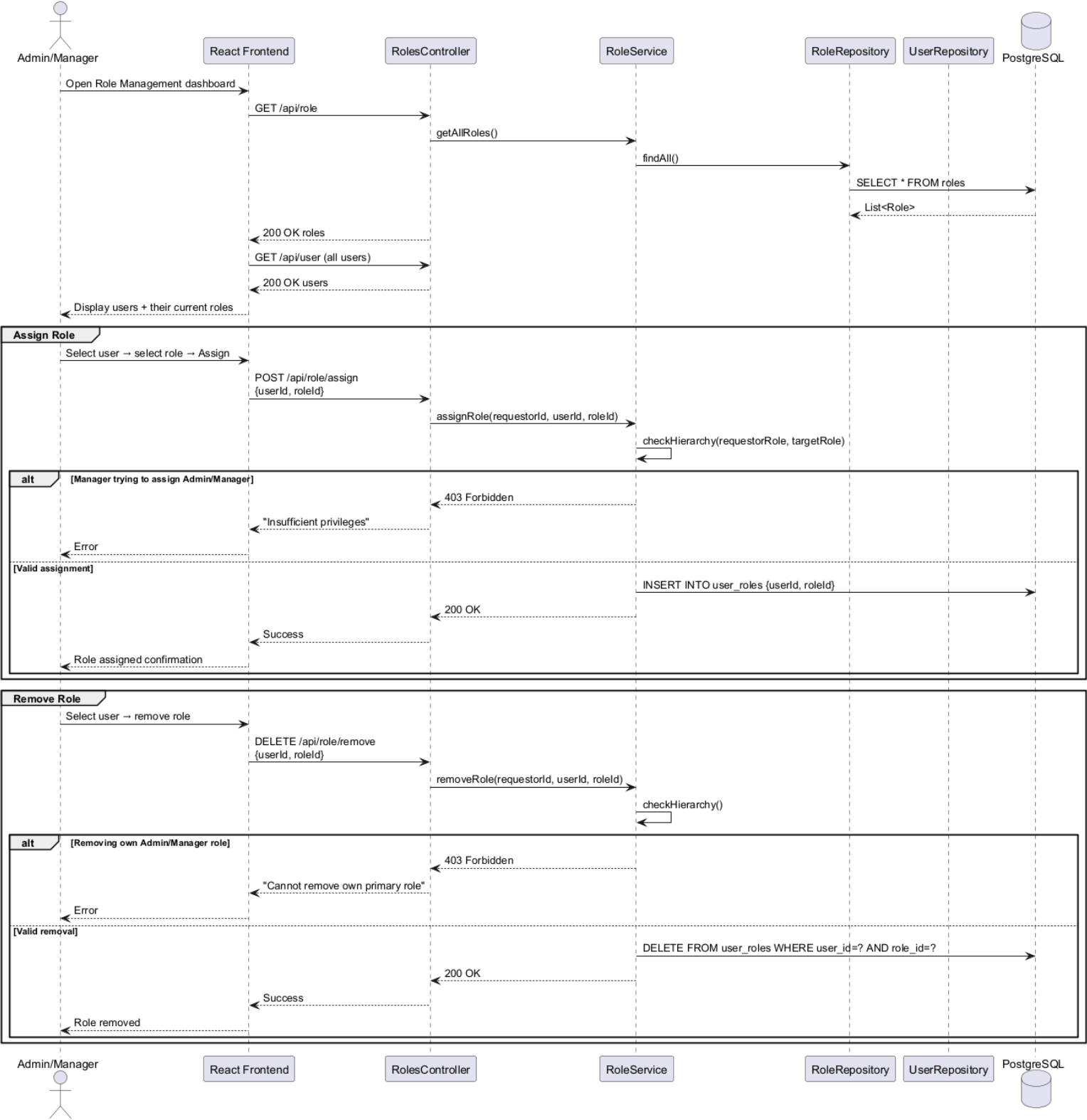
U-022 · Manage Services (Admin / Manager)



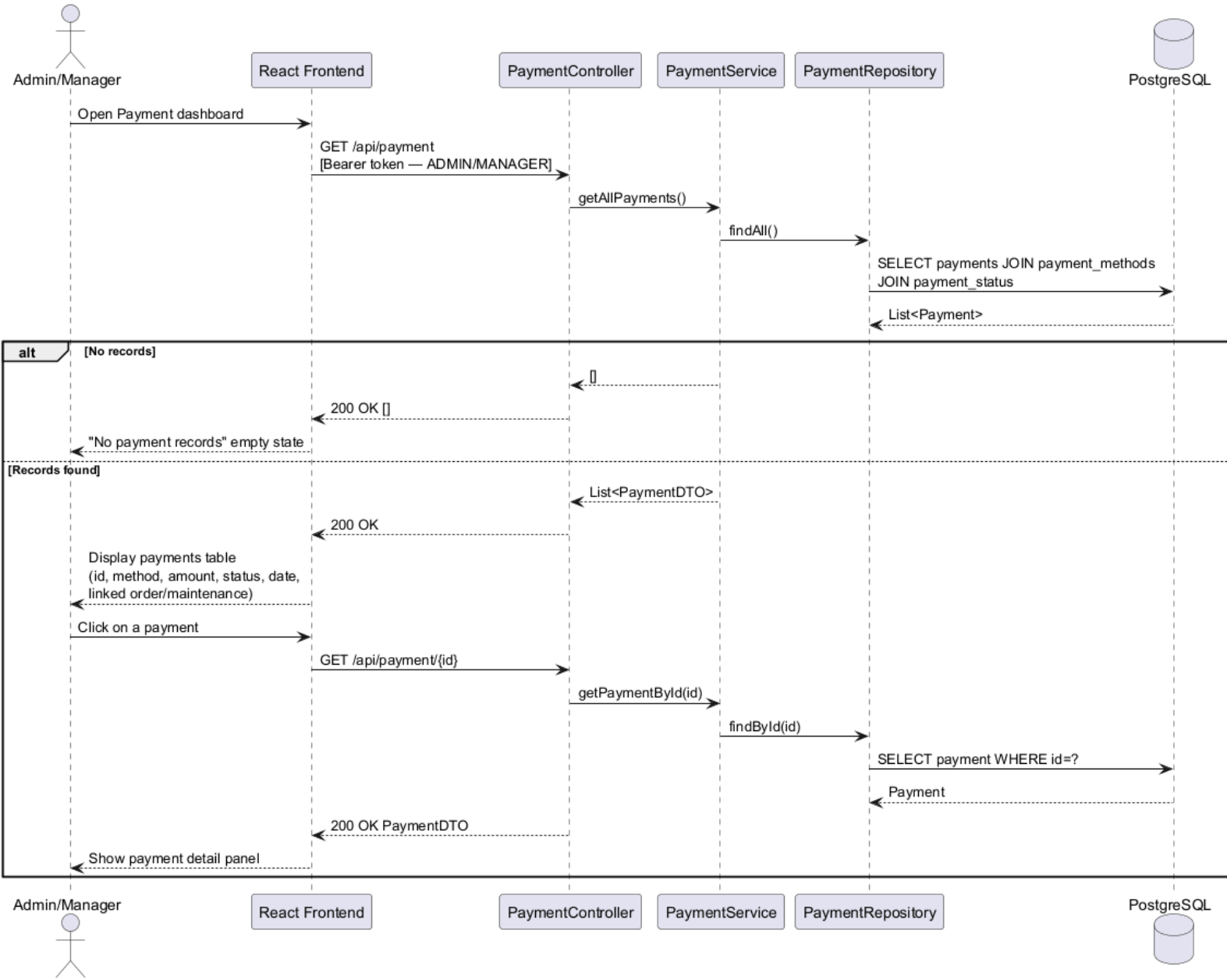
U-023 · Manage Payment Methods (Admin / Manager)



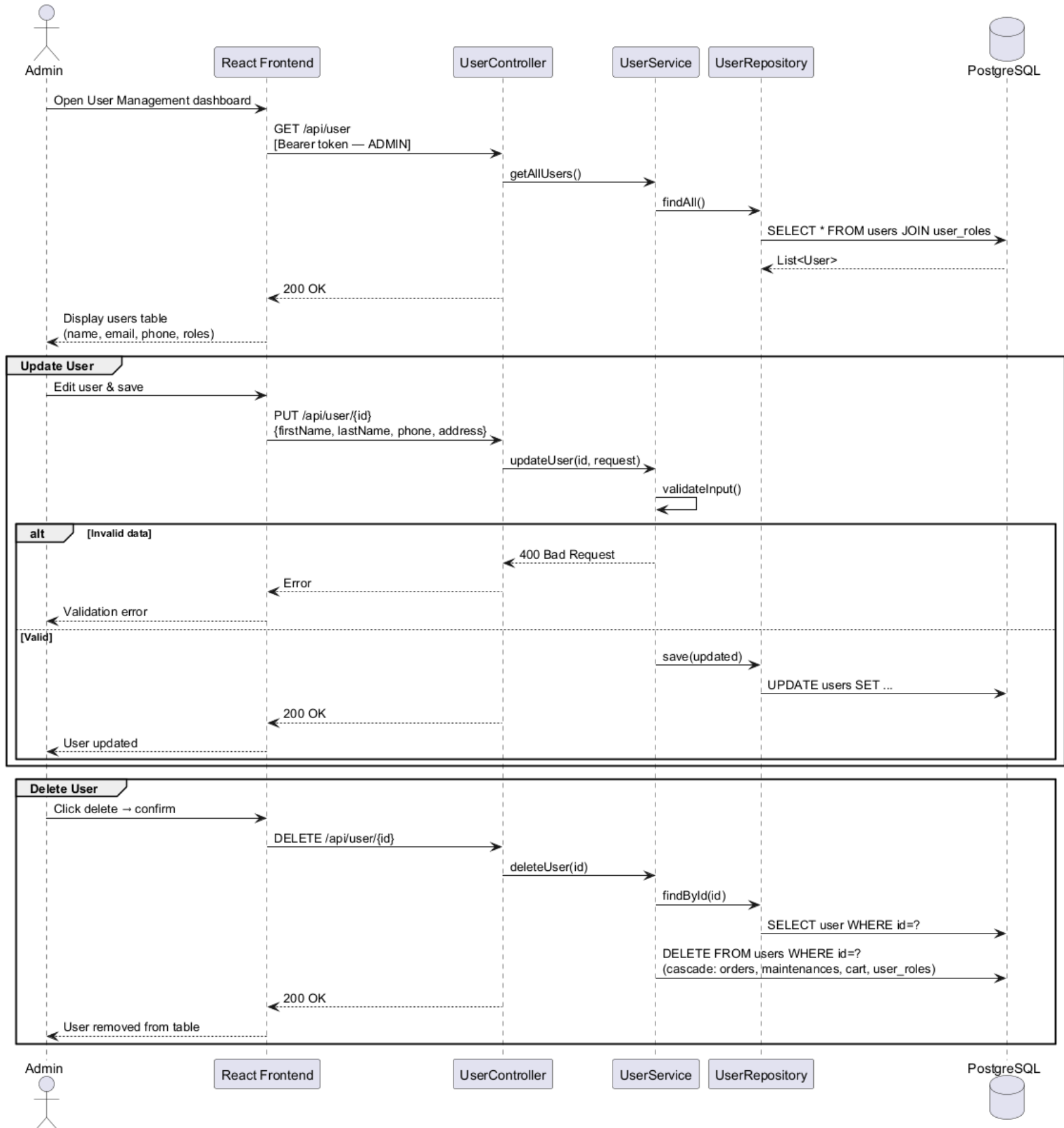
U-024 · Manage User Roles (Admin / Manager)



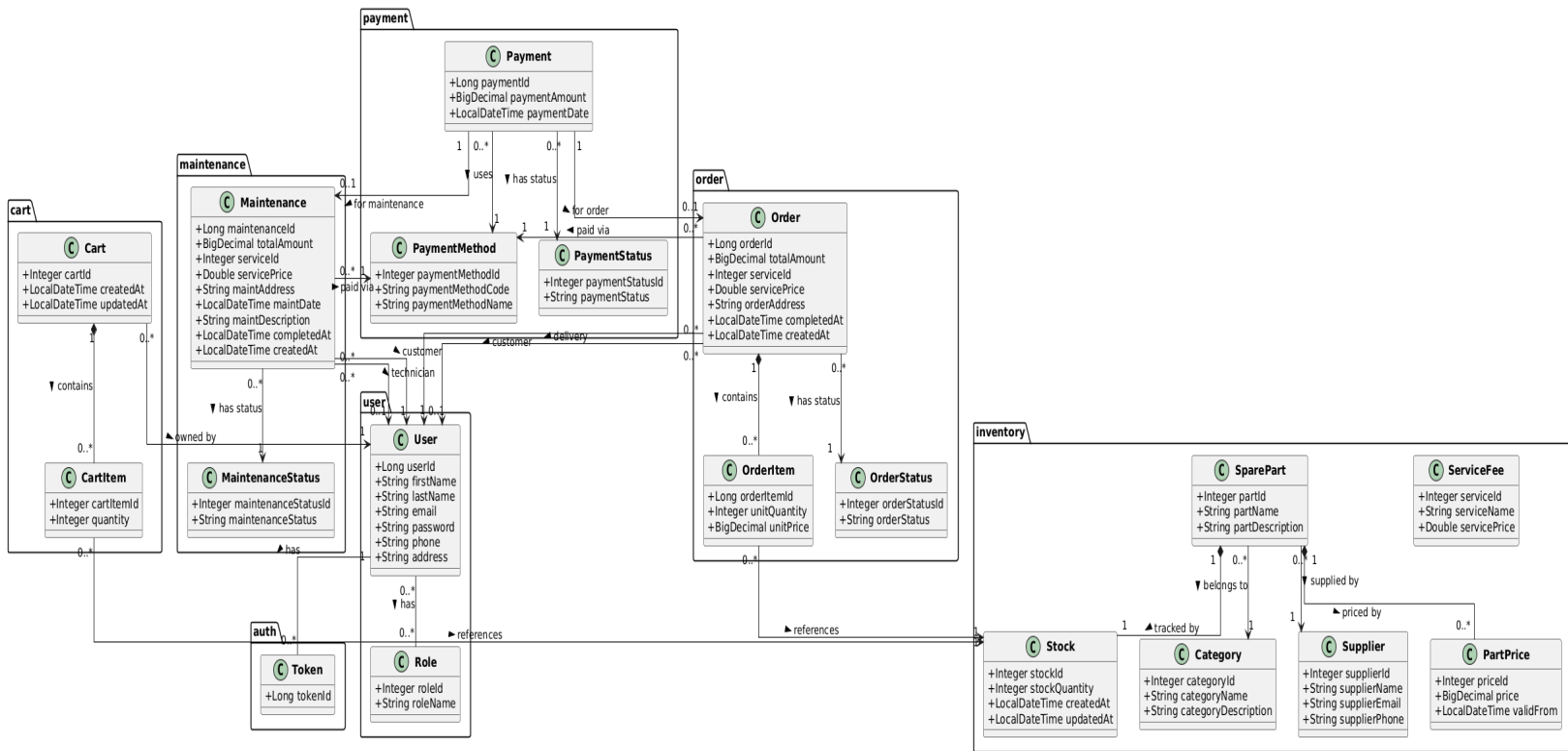
U-025 · View Payments (Admin / Manager)



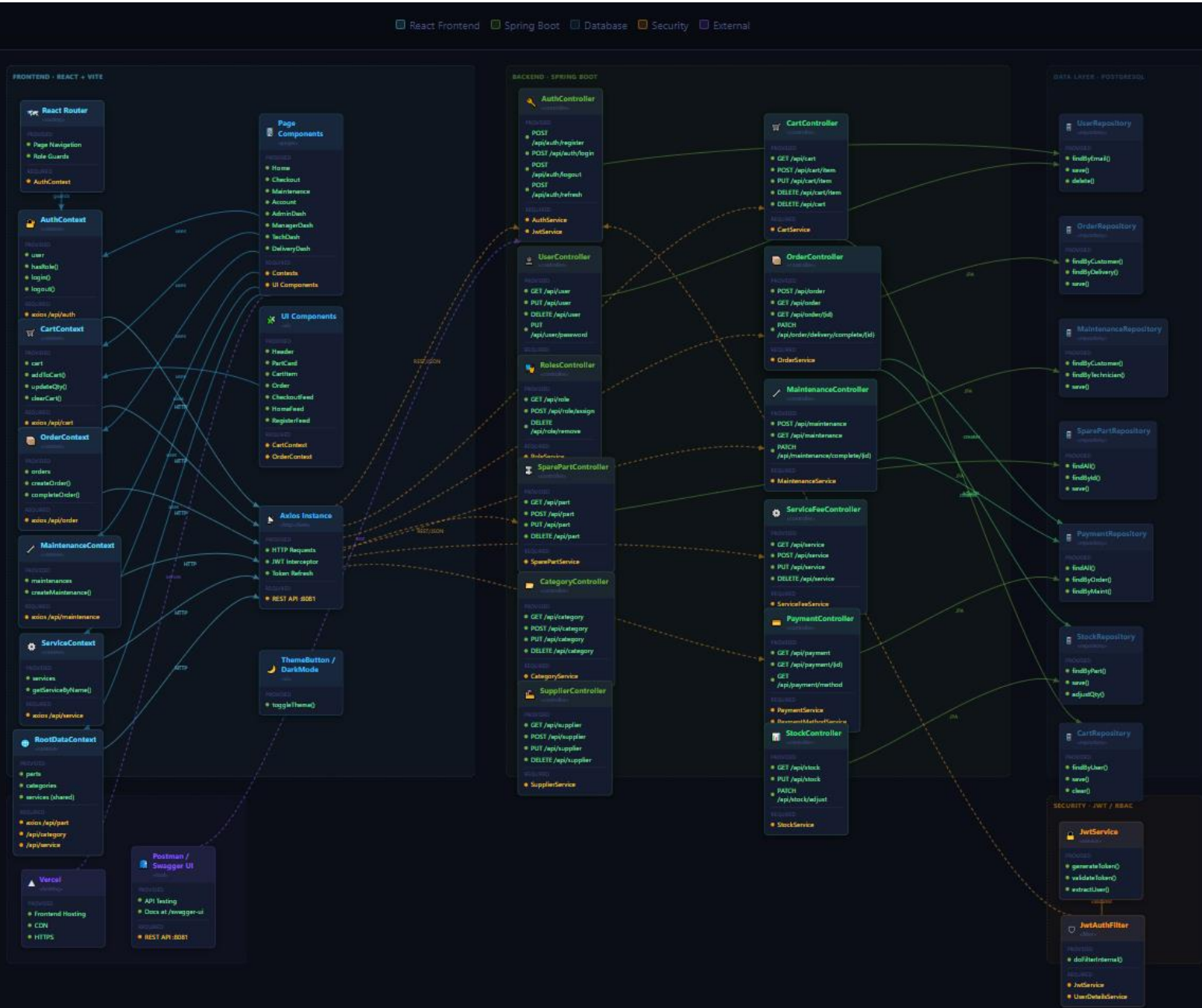
U-026 · Manage Users (Admin only)



6.3 Class Diagram:



6.4 Component Diagram:



7. System Architecture

7.1 Overview

The system architecture is designed using a three-tier architecture that separates the application into presentation, application, and data layers. This design improves maintainability, scalability, and system organization.

The frontend is developed using ReactJS and communicates with a backend built using Spring Boot through RESTful APIs. The backend processes requests, applies business logic, and interacts with a PostgreSQL database for persistent storage.

The system supports multiple roles including Customer, Technician, Delivery, Manager, and Administrator, with role-based access control enforced at the backend level.

7.2 Architecture Layers

7.2.1 Data Layer (PostgreSQL)

The data layer uses PostgreSQL to store structured relational data. It supports all system entities and their relationships.

Key characteristics:

- Strong referential integrity using foreign keys
- Transaction support (ACID compliance)
- Efficient handling of relationships between entities
- Support for historical tracking such as:
 - Price history for spare parts
 - Order and maintenance records
 - Payment status tracking

7.2.2 Presentation Layer (Frontend)

The presentation layer is implemented using ReactJS and is responsible for delivering an interactive and responsive user interface.

The frontend includes multiple role-based dashboards, such as:

- Customer Dashboard (cart, orders, maintenance booking)
- Admin Dashboard (user, role, and system management)
- Manager Dashboard (inventory, categories, suppliers)
- Technician Dashboard (assigned maintenance tasks)
- Delivery Dashboard (assigned deliveries)

Key functionalities include:

- Form handling and validation
 - State management and UI updates
 - API communication using HTTP requests
 - Dynamic rendering based on user role
-

7.2.3 Application Layer (Backend – Spring Boot)

The backend is built using Spring Boot and follows a layered architecture consisting of Controllers, Services, and Repositories.

Controller Layer

The controller layer exposes RESTful endpoints and handles incoming HTTP requests. Each controller is responsible for a specific module in the system.

Main controllers include:

- **AuthController** → Handles user registration, login, and JWT authentication
- **UserController** → Manages user operations (view, update, delete)
- **RolesController** → Handles role retrieval and assignment
- **SupplierController** → Manages suppliers

- **CategoryController** → Handles product categories
- **ServiceFeeController** → Manages service definitions and associated pricing.
- **SparePartController** → Handles spare part operations
- **PartPriceController** → Manages spare part price history and retrieval of current prices
- **StockController** → Manages inventory levels
- **CartController** → Handles cart operations
- **OrderController** → Processes orders and checkout
- **MaintenanceController** → Manages maintenance appointments
- **PaymentController** → Handles payment operations
- **PaymentMethodController** → Manages available payment methods

These controllers map HTTP methods (GET, POST, PUT, DELETE) to system functionalities.

Service Layer

The service layer contains the core business logic of the application. It processes data, enforces rules, and coordinates between controllers and repositories.

Main services include:

- **AuthService** → Handles authentication and token generation
- **RefreshTokenService** →
- **UserService** → Business logic for user management
- **RoleService** → Role assignment and hierarchy enforcement
- **SupplierService** → Supplier operations
- **CategoryService** → Category management
- **ServiceFeeService** → Handles business logic related to services pricing.
- **SparePartService** → Product handling and validation
- **PartPriceService** → Manages price history for spare parts, including adding new price records and retrieving current or historical prices

- **StockService** → Stock updates and adjustments
- **CartService** → Cart operations and validation
- **OrderService** → Checkout logic and order processing
- **MaintenanceService** → Appointment scheduling and status updates
- **PaymentService** → Payment validation and tracking
- **PaymentMethodService** → Manages payment method creation, updates, deletion, and retrieval

Key responsibilities:

- Enforcing **role-based access control (RBAC)**
- Handling **price snapshots** for orders and services
- Managing **cart-to-order conversion**
- Assigning technicians and delivery personnel
- Validating business rules and constraints

Repository Layer

The repository layer interacts directly with the PostgreSQL database using JPA/Hibernate.

Each entity has a corresponding repository, such as:

- TokenRepository
- RefreshTokenRepository
- UserRepository
- RoleRepository
- SparePartRepository
- CategoryRepository
- SupplierRepository
- StockRepository
- ServiceFeeRepository

- PartPriceRepository
- OrderRepository
- OrderItemRepository
- OrderStatusRepository
- CartRepository
- CartItemRepository
- MaintenanceRepository
- MaintenanceStatusRepository
- PaymentRepository
- PaymentMethodRepository
- PaymentStatusRepository

Responsibilities include:

- CRUD operations
- Query execution
- Data persistence and retrieval

7.3 Security Architecture

Security is a core aspect of the system and is implemented using **JWT (JSON Web Tokens)**.

- Users authenticate via the **AuthController**
- A JWT token is generated upon successful login
- The token is included in request headers for protected endpoints
- Backend validates the token before granting access

Additionally, **role-based access control (RBAC)** ensures that users can only access functionalities permitted by their roles.

7.4 System Communication

The system components communicate through RESTful APIs:

- Frontend → Backend via HTTP (JSON format)
- Backend → Database via JPA/Hibernate

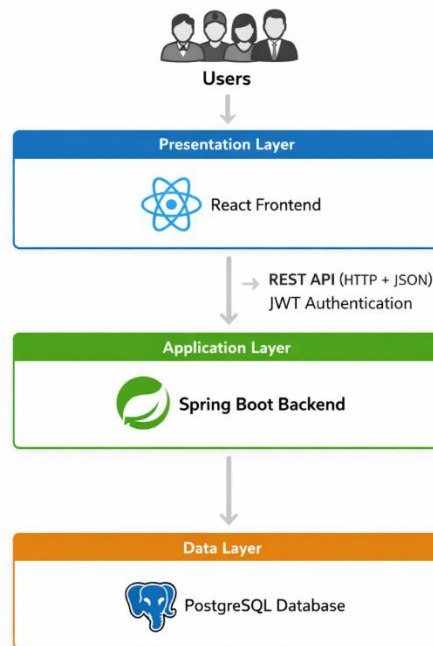
Each request follows this flow:

1. User interacts with frontend
 2. Frontend sends API request
 3. Controller receives request
 4. Service processes logic
 5. Repository interacts with database
 6. Response returned to frontend
-

7.5 Architectural Advantages

- Clear separation of concerns through layered architecture
- Secure access control using JWT and RBAC
- Scalable structure supporting multiple roles and modules
- Maintainable codebase with modular services and controllers
- Efficient handling of complex operations such as orders and maintenance

7.6 Architecture Diagram



8. Security & Roles

8.1 Authentication (JWT-Based Security)

The system implements authentication using **JSON Web Tokens (JWT)** to ensure secure access for all users. When a user logs in with valid credentials, the backend generates a JWT token that contains encoded user information such as user email.

This token is sent to the client and stored securely (typically in local storage or cookies). For every subsequent request, the token is included in the request headers to verify the user's identity.

The backend validates the token signature and expiration before granting access to protected endpoints. If the token is invalid or expired, the user is redirected to the login page.

This mechanism ensures stateless authentication, improving both scalability and security.

8.2 Authorization (Role-Based Access Control)

The system uses **Role-Based Access Control (RBAC)** to restrict access to features based on user roles.

Each user is assigned a specific role:

- Admin
- Manager
- Technician
- Delivery
- Customer

Each role has predefined permissions:

- **Admin:** Full system access including user management, roles, inventory, payments, and system configuration.
- **Manager:** Manages operational data such as products, categories, suppliers, stock, and services.
- **Technician:** Access to assigned maintenance appointments only.
- **Delivery:** Access to assigned orders for delivery only.
- **Customer:** Access to personal account, cart, orders, and maintenance booking.

The system checks the user role from the JWT token before granting access to specific endpoints or UI components.

9. System Features

The system provides a comprehensive platform that integrates e-commerce functionality, maintenance services, and administrative control.

9.1 E-Commerce System

The platform includes a complete e-commerce module that allows customers to browse spare parts, add items to a cart, and place orders. The system supports:

- Product browsing and filtering
 - Shopping cart management
 - Order creation and history tracking
 - Secure payment processing
 - Price snapshot to maintain historical accuracy
-

9.2 Maintenance Appointment System

Customers can schedule maintenance appointments for printer servicing. The system:

- Allows users to book maintenance requests
 - Stores service details including address and description
 - Automatically assigns available technicians
 - Tracks appointment status (Scheduled, Completed)
-

9.3 Role-Based Dashboards

Each user role has a dedicated dashboard providing relevant functionality:

- **Admin Dashboard:** Full system control and monitoring
- **Manager Dashboard:** Inventory and operational management
- **Technician Dashboard:** Assigned maintenance tasks
- **Delivery Dashboard:** Assigned delivery orders
- **Customer Dashboard:** Orders, cart, and maintenances overview

9.4 Inventory Management System

The system includes a full inventory control module that manages:

- Spare parts catalog
- Categories and suppliers
- Stock levels and adjustments
- Part price history tracking
- Automatic stock creation when new parts are added

This ensures accurate and real-time inventory tracking across the system.

10. Testing

The system was tested using multiple testing strategies to ensure reliability, performance, and correctness.

10.1 Unit Testing

Unit testing was performed on individual backend components such as services, controllers, and utility functions. The goal was to ensure that each function behaves correctly in isolation.

Examples include:

- User authentication service
- Cart and order logic
- Stock update functions

10.2 API Testing

API endpoints were tested using tools such as Postman or similar testing frameworks. This ensured that:

- All endpoints return correct responses
 - Authentication and authorization work properly
 - CRUD operations function as expected
 - Error handling is properly implemented
-

10.3 UI Testing

The React frontend was tested to ensure usability and correct integration with backend APIs.

Testing included:

- Form validation (login, register, checkout)
 - Navigation between pages
 - Role-based rendering of components
 - Dashboard functionality for each role
-

10.4 API Testing Tool (Postman)

The backend REST APIs were tested using Postman, which allowed verification of all HTTP requests such as GET, POST, PUT, and DELETE.

Postman was used to:

- Test authentication endpoints (login/register with JWT)
- Validate CRUD operations for users, products, orders, and maintenance
- Verify role-based access control responses
- Ensure correct request/response formats
- Debug API errors during development

11. Conclusion

11.1 Project Achievements

This project successfully delivered a full-stack web application for managing printer spare parts and maintenance services. The system integrates:

- A secure authentication and authorization system using JWT and RBAC
- A complete e-commerce platform with cart, order, and payment functionality
- A maintenance management system with automated technician assignment
- A structured inventory management system with stock tracking and price history
- Role-based dashboards for efficient user interaction

11.2 Challenges

During development, several challenges were encountered:

- Designing a scalable role-based architecture
- Ensuring data consistency between orders, stock, and payments
- Implementing automatic assignment logic for technicians and delivery personnel
- Integrating frontend React components with backend Spring Boot APIs
- Managing state consistency in cart and order workflows

11.3 Future Improvements

The system can be further enhanced by adding:

- Real-time notifications for orders and maintenance updates
- AI-based spare part recommendations
- Advanced analytics dashboard for sales and inventory trends
- Mobile application support
- Live tracking for delivery orders
- Enhanced payment gateway integration